Wi-Fi Project File Export

Generated by Copilot assistant

backend files

backend/alembic.ini

```
# A generic, single database configuration.

[alembic]
# path to migration scripts.
# this is typically a path given in POSIX (e.g. forward slashes)
# format, relative to the token %(here)s which refers to the location of this
# ini file
script_location = %(here)s/migrations

# template used to generate migration file names; The default value is %%(rev)s_%%(slug)s
# Uncomment the line below if you want the files to be prepended with date and time
# see https://alembic.sqlalchemy.org/en/latest/tutorial.html#editing-the-ini-file
# for all available tokens
# file_template = %%(year)d_%%(month).2d_%%(day).2d_%%(hour).2d%%(minute).2d-%%(rev)s_%%(slug)s
# Or organize into date-based subdirectories (requires recursive_version_locations = true)
# file_template = %%(year)d/%%(month).2d/%%(day).2d_%%(hour).2d%%(minute).2d_%%(second).2d_%%(rev)s_

# sys.path path, will be prepended to sys.path if present.
# defaults to the current working directory.  for multiple paths, the path separator
# is defined by "path_separator" below.
prepend_sys_path = .

# timezone to use when rendering the date within the migration file
# as well as the filename.
# If specified, requires the tzdata library which can be installed by adding
# `alembic[tz]` to the pip requirements.
# string value is passed to ZoneInfo()
# leave blank for localtime
# timezone =

# max length of characters to apply to the "slug" field
# truncate_slug_length = 40

# set to 'true' to run the environment during
# the 'revision' command, regardless of autogenerate
# revision_environment = false

# set to 'true' to allow .pyc and .pyo files without
# a source .py file to be detected as revisions in the
# versions/ directory
# sourceless = false

# version location specification; This defaults
# to <script_location>/versions.  When using multiple version
# directories, initial revisions must be specified with --version-path.
# The path separator used here should be the separator specified by "path_separator"
# below.
# version_locations = %(here)s/bar:%(here)s/bat:%(here)s/alembic/versions

# path_separator; This indicates what character is used to split lists of file
# paths, including version_locations and prepend_sys_path within configparser
# files such as alembic.ini.
# The default rendered in new alembic.ini files is "os", which uses os.pathsep
# to provide os-dependent path splitting.
#
# Note that in order to support legacy alembic.ini files, this default does NOT
# take place if path_separator is not present in alembic.ini.  If this
# option is omitted entirely, fallback logic is as follows:
#
```

```
# 1. Parsing of the version_locations option falls back to using the legacy
#    "version_path_separator" key, which if absent then falls back to the legacy
#    behavior of splitting on spaces and/or commas.
# 2. Parsing of the prepend_sys_path option falls back to the legacy
#    behavior of splitting on spaces, commas, or colons.
#
# Valid values for path_separator are:
#
# path_separator = :
# path_separator = ;
# path_separator = space
# path_separator = newline
#
# Use os.pathsep. Default configuration used for new projects.
path_separator = os


# set to 'true' to search source files recursively
# in each "version_locations" directory
# new in Alembic version 1.10
# recursive_version_locations = false

# the output encoding used when revision files
# are written from script.py.mako
# output_encoding = utf-8

# database URL.  This is consumed by the user-maintained env.py script only.
# other means of configuring database URLs may be customized within the env.py
# file.
sqlalchemy.url = driver://user:pass@localhost/dbname


[post_write_hooks]
# post_write_hooks defines scripts or Python functions that are run
# on newly generated revision scripts.  See the documentation for further
# detail and examples

# format using "black" - use the console_scripts runner, against the "black" entrypoint
# hooks = black
# black.type = console_scripts
# black.entrypoint = black
# black.options = -l 79 REVISION_SCRIPT_FILENAME

# lint with attempts to fix using "ruff" - use the module runner, against the "ruff" module
# hooks = ruff
# ruff.type = module
# ruff.module = ruff
# ruff.options = check --fix REVISION_SCRIPT_FILENAME

# Alternatively, use the exec runner to execute a binary found on your PATH
# hooks = ruff
# ruff.type = exec
# ruff.executable = ruff
# ruff.options = check --fix REVISION_SCRIPT_FILENAME

# Logging configuration.  This is also consumed by the user-maintained
# env.py script only.
[loggers]
keys = root,sqlalchemy,alembic

[handlers]
keys = console

[formatters]
keys = generic

[logger_root]
level = WARNING
```

```
handlers = console
qualname =

[logger_sqlalchemy]
level = WARNING
handlers =
qualname = sqlalchemy.engine

[logger_alembic]
level = INFO
handlers =
qualname = alembic

[handler_console]
class = StreamHandler
args = (sys.stderr,)
level = NOTSET
formatter = generic

[formatter_generic]
format = %(levelname)-5.5s [%(name)s] %(message)s
datefmt = %H:%M:%S
```

backend/app/api/dependencies/auth.py

```
import uuid
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
import jwt

from app.core.security import SECRET_KEY, ALGORITHM
from app.db.session import get_db
from app.db.models import User

# This tells FastAPI
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/v1/auth/login")

async def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: AsyncSession = Depends(get_db)
) -> User:
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )

    try:
        # Decode the JWT
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id: str = payload.get("sub")

        if user_id is None:
            raise credentials_exception

    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail="Token has expired")
    except jwt.InvalidTokenError:
        raise credentials_exception

    # Verify user still exists in the database
    stmt = select(User).where(User.id == uuid.UUID(user_id))
    result = await db.execute(stmt)
```

```
    user = result.scalar_one_or_none()

    if user is None:
        raise credentials_exception

    return user

async def get_current_tenant(current_user: User = Depends(get_current_user)) -> uuid.UUID:
    """
    Extracts the tenant ID from the authenticated user.
    Use this dependency in all multi-tenant endpoints to scope database queries.
    """
    if not current_user.tenant_id:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="User does not belong to a valid tenant organization."
        )
    return current_user.tenant_id

class RoleChecker:
    """
    Dependency class to enforce Role-Based Access Control.
    """
    def __init__(self, allowed_roles: list[str]):
        self.allowed_roles = allowed_roles

    def __call__(self, current_user: User = Depends(get_current_user)):
        if current_user.role not in self.allowed_roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"Operation not permitted. Required roles: {', '.join(self.allowed_roles)}"
            )
        return current_user
```

backend/app/api/dependencies/tenant.py

```
# security
from fastapi import Depends, HTTPException, status
from sqlalchemy.ext.asyncio import AsyncSession
from app.db.session import get_db
from app.api.dependencies.auth import get_current_user
from app.db.models import User

async def get_current_tenant_id(
    current_user: User = Depends(get_current_user)
) -> str:
    """
    Extracts the tenant_id from the authenticated user.
    Ensures that all subsequent DB queries are scoped to this tenant.
    """
    if not current_user.tenant_id:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="User does not belong to a valid tenant context."
        )
    return current_user.tenant_id
```

backend/app/api/deps.py

```
from app.core.config import settings

def get_current_tenant() -> str:
    """
    Overrides multi-tenant logic.
```

```
    Forces every API call to map to the single local instance.
    """
    return settings.LOCAL_TENANT_ID
```

backend/app/api/v1/endpoints/admin.py

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select, func, desc

# Database dependencies & Models
from app.db.session import get_db
from app.db.models import Transaction, Plan, Tenant

router = APIRouter()

# 1. DASHBOARD STATISTICS
@router.get("/dashboard/stats/{tenant_id}")
async def get_dashboard_stats(tenant_id: str, db: AsyncSession = Depends(get_db)):
    """
    Fetches high-level metrics: Total Revenue and Total Successful Transactions.
    """
    # Query 1: Calculate Total Revenue (Only sum COMPLETED transactions)
    revenue_stmt = select(func.sum(Transaction.amount)).where(
        Transaction.tenant_id == tenant_id,
        Transaction.status == "COMPLETED"
    )
    revenue_result = await db.execute(revenue_stmt)
    total_revenue = revenue_result.scalar() or 0.0

    # Query 2: Count Total Successful Connections
    sales_stmt = select(func.count(Transaction.id)).where(
        Transaction.tenant_id == tenant_id,
        Transaction.status == "COMPLETED"
    )
    sales_result = await db.execute(sales_stmt)
    total_sales = sales_result.scalar() or 0

    return {
        "total_revenue_kes": total_revenue,
        "total_sales": total_sales,
        "active_users": total_sales # We will refine this later to only count unexpired plans
    }


# 2. TRANSACTION HISTORY
@router.get("/transactions/{tenant_id}")
async def get_all_transactions(tenant_id: str, db: AsyncSession = Depends(get_db)):
    """
    Fetches the raw history of all M-Pesa attempts (Completed, Pending, and Failed)
    ordered by the newest first.
    """
    stmt = select(Transaction).where(
        Transaction.tenant_id == tenant_id
    ).order_by(desc(Transaction.created_at)) # Assuming you have a created_at timestamp column

    result = await db.execute(stmt)
    transactions = result.scalars().all()

    # Convert to a list of dictionaries for React to consume easily
    tx_list = []
    for tx in transactions:
        tx_list.append({
            "id": str(tx.id),
            "amount": tx.amount,
```

```
            "status": tx.status,
            "checkout_request_id": tx.checkout_request_id,
            # If you added phone_number to your Transaction model, include it here:
            # "phone_number": tx.phone_number
        })

    return {"transactions": tx_list}
```

backend/app/api/v1/endpoints/auth.py

```
from datetime import timedelta
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select

from app.db.session import get_db
from app.db.models import User
from app.core.security import verify_password, create_access_token, ACCESS_TOKEN_EXPIRE_MINUTES

router = APIRouter()

@router.post("/login")
async def login_access_token(
    db: AsyncSession = Depends(get_db),
    form_data: OAuth2PasswordRequestForm = Depends()
):
    """
    OAuth2 compatible token login, getting an access token for future requests.
    Note: OAuth2PasswordRequestForm expects data as form-urlencoded, not JSON.
    """
    # 1. Find the user by email
    stmt = select(User).where(User.email == form_data.username)
    result = await db.execute(stmt)
    user = result.scalar_one_or_none()

    # 2. Verify existence and password
    if not user or not verify_password(form_data.password, user.password_hash):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect email or password",
        )

    # 3. Create JWT with user identity and tenant context
    access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)

    token_payload = {
        "sub": str(user.id),
        "tenant_id": str(user.tenant_id) if user.tenant_id else None,
        "role": user.role
    }

    access_token = create_access_token(
        data=token_payload, expires_delta=access_token_expires
    )

    return {
        "access_token": access_token,
        "token_type": "bearer",
        "user_info": {
            "email": user.email,
            "role": user.role
        }
    }
```

backend/app/api/v1/endpoints/customers.py

```

```
from fastapi import APIRouter, Depends
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from app.db.session import get_db
from app.api.dependencies.tenant import get_current_tenant_id
from app.db.models import Customer

router = APIRouter()

@router.post("/customers/")
async def create_local_customer(
    customer_data: dict, # Replace with your Pydantic schema
    db: AsyncSession = Depends(get_db),
    tenant_id: str = Depends(deps.get_current_tenant) #THE LOCK that forces the system to use the ha
e on the config file
):
    # This willexplicitly attach the locked tenant_id to the incoming data
    new_customer = Customer(
        tenant_id=tenant_id,
        phone_number=customer_data['phone_number'],
        mac_address=customer_data.get('mac_address'),
        status="ACTIVE"
)

    db.add(new_customer)
    await db.commit()
    await db.refresh(new_customer)
    return new_customer
```

### backend/app/api/v1/endpoints/mpesa.py

```
import os
import uuid
import secrets
from fastapi import APIRouter, Request, BackgroundTasks, Depends, HTTPException, status
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select

# Database dependencies & Models
from app.db.session import get_db
from app.db.models import Tenant, Plan, Transaction

# Domain Services
from app.services.mpesa_service import DarajaService
from app.services.mikrotik_service import activate_customer_on_router

from pydantic import BaseModel

class STKPushRequest(BaseModel):
    phone_number: str
    plan_id: str
    tenant_id: str

router = APIRouter()

1. BACKGROUND TASK WORKER (INTERNAL)
async def process_successful_payment(
    tenant_id: str,
    phone: str,
    plan_id: str
):
    """
```
```

```
    Decoupled background worker. Executes concurrently *after* FastAPI replies '200 OK'.
    """
    try:
        # We now pass plan_id so the Mikrotik service knows WHICH speed profile to assign
        await activate_customer_on_router(tenant_id=tenant_id, phone=phone, plan_id=plan_id)
        print(f"■ Background task complete: Provisioned {phone} on Tenant {tenant_id}")
    except Exception as e:
        print(f"■ Critical background processing error: {str(e)}")


# 2. OUTBOUND: INITIATE PAYMENT (FROM CLIENT)
@router.post("/stk-push")
async def trigger_payment_request(
    payload: STKPushRequest,
    db: AsyncSession = Depends(get_db)
):
    """
    Called by your Captive Portal when a user hits 'Pay via M-Pesa'.
    """
    phone = payload.phone_number
    plan_id = payload.plan_id
    tenant_id = payload.tenant_id

    # Fetch tenant-specific Daraja credentials
    tenant_stmt = select(Tenant).where(Tenant.id == uuid.UUID(tenant_id))
    tenant_result = await db.execute(tenant_stmt)
    tenant = tenant_result.scalar_one_or_none()
    if not tenant:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Tenant ISP not found")

    # Fetch the plan details
    plan_stmt = select(Plan).where(Plan.id == uuid.UUID(plan_id))
    plan_result = await db.execute(plan_stmt)
    plan = plan_result.scalar_one_or_none()
    if not plan:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Selected Internet Plan no

    # Instantiate Daraja Service (Use env vars for global configs)
    mpesa_engine = DarajaService(
        consumer_key=tenant.daraja_consumer_key,
        consumer_secret=tenant.daraja_consumer_secret,
        shortcode=tenant.daraja_shortcode or os.getenv("DARAJA_DEFAULT_SHORTCODE", "174379"),
        passkey=os.getenv("DARAJA_PASSKEY")
    )

    # Dynamically build webhook URL. Use env var in production, default to ngrok locally.
    webhook_domain = os.getenv("WEBHOOK_DOMAIN", "https://winfred-uncaned-jerri.ngrok-free.dev")
    callback_endpoint = f"{webhook_domain}/api/v1/mpesa/callback/{tenant_id}"

    # Send request to Safaricom API
    result = await mpesa_engine.initiate_stk_push(
        phone_number=phone,
        amount=int(plan.price),
        callback_url=callback_endpoint,
        account_reference=phone,
        transaction_desc=f"WiFi {plan.name}"
    )

    if not result.get("success"):
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail=result.get("error", "STK
ed"))

    # Save a PENDING record WITH the plan_id
    new_tx = Transaction(
        tenant_id=tenant.id,
        plan_id=plan.id,
        amount=plan.price,
        checkout_request_id=result["checkout_request_id"],
```

```
        status="PENDING"
    )
    db.add(new_tx)
    await db.commit()

    return {"message": "STK Push sent to phone", "checkout_id": result["checkout_request_id"]}


# 3. INBOUND: WEBHOOK CALLBACK (FROM SAFARICOM)
@router.post("/callback/{tenant_id}")
async def stk_push_callback(
    tenant_id: str,
    request: Request,
    background_tasks: BackgroundTasks,
    db: AsyncSession = Depends(get_db)
):
    """
    Safaricom calls this endpoint asynchronously when the user completes or cancels PIN entry.
    """
    payload = await request.json()
    stk_callback = payload.get("Body", {}).get("stkCallback", {})
    result_code = stk_callback.get("ResultCode")
    checkout_request_id = stk_callback.get("CheckoutRequestID")

    stmt = select(Transaction).where(Transaction.checkout_request_id == checkout_request_id)
    result = await db.execute(stmt)
    transaction = result.scalar_one_or_none()

    if not transaction:
        print(f"■■ Orphaned Webhook: Checkout {checkout_request_id} not found.")
        return {"ResultCode": 0, "ResultDesc": "Acknowledged but missing locally"}

    if result_code == 0:
        meta = stk_callback.get("CallbackMetadata", {}).get("Item", [])
        phone = next((i["Value"] for i in meta if i["Name"] == "PhoneNumber"), None)
        receipt = next((i["Value"] for i in meta if i["Name"] == "MpesaReceiptNumber"), None)

        # 1. Update State
        transaction.status = "COMPLETED"
        transaction.mpesa_receipt = receipt

        # 2. Generate the Zero-Cost Transfer PIN (6-digit alphanumeric string)
        transfer_pin = secrets.token_hex(3).upper()
        transaction.transfer_pin = transfer_pin

        await db.commit()
        print(f"■ Payment {receipt} recorded. PIN generated: {transfer_pin}")

        # 3. Hand off to the MikroTik background worker
        background_tasks.add_task(
            process_successful_payment,
            tenant_id=tenant_id,
            phone=str(phone),
            plan_id=str(transaction.plan_id)
        )
    else:
        transaction.status = "FAILED"
        await db.commit()
        print(f"■ M-Pesa transaction {checkout_request_id} failed with code {result_code}")

    return {"ResultCode": 0, "ResultDesc": "Callback Processed Successfully"}


# 4. INBOUND: STATUS POLLING (FROM CLIENT)
@router.get("/status/{checkout_request_id}")
async def check_transaction_status(
    checkout_request_id: str,
    db: AsyncSession = Depends(get_db)
```

```
):
    """
    Called by the React frontend polling engine every 3 seconds.
    """
    stmt = select(Transaction).where(Transaction.checkout_request_id == checkout_request_id)
    result = await db.execute(stmt)
    transaction = result.scalar_one_or_none()

    if not transaction:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Transaction not found in the system."
        )

    return {
        "status": transaction.status,
        "receipt": transaction.mpesa_receipt,
        "transfer_pin": transaction.transfer_pin
    }
```

backend/app/api/v1/endpoints/routers.py

```
# app/api/v1/endpoints/routers.py
from fastapi import APIRouter, Depends
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
import uuid

from app.db.session import get_db
from app.db.models import Router
from app.api.dependencies.auth import get_current_tenant, RoleChecker

router = APIRouter()

# Define allowed roles for certain actions
allow_admin_only = RoleChecker(["TENANT_ADMIN", "SUPER_ADMIN"])
allow_all_staff = RoleChecker(["TENANT_ADMIN", "RESELLER", "SUPER_ADMIN"])

@router.get("/")
async def list_routers(
    db: AsyncSession = Depends(get_db),
    tenant_id: uuid.UUID = Depends(get_current_tenant),
    user_access = Depends(allow_all_staff)
):
    """
    Any staff member can view the routers, but ONLY the routers belonging to their specific tenant_i
    """
    stmt = select(Router).where(Router.tenant_id == tenant_id)
    result = await db.execute(stmt)
    return result.scalars().all()

@router.delete("/{router_id}")
async def delete_router(
    router_id: uuid.UUID,
    db: AsyncSession = Depends(get_db),
    tenant_id: uuid.UUID = Depends(get_current_tenant),
    user_access = Depends(allow_admin_only) # REJECTS RESELLERS
):
    """
    Only Admins can delete routers. The query strictly includes tenant_id
    to prevent an admin from deleting another ISP's router by guessing the UUID.
    """
    stmt = select(Router).where(Router.id == router_id, Router.tenant_id == tenant_id)
    result = await db.execute(stmt)
    router_obj = result.scalar_one_or_none()
```

```
    if not router_obj:
        raise HTTPException(status_code=404, detail="Router not found")

    await db.delete(router_obj)
    await db.commit()
    return {"message": "Router deleted successfully"}
```

backend/app/api/v1/endpoints/tenants.py

```

```

backend/app/core/config.py

```
import os
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    PROJECT_NAME: str = "Wi-Fi Billing System (Standalone)"
    SECRET_KEY: str = "change-this-super-secret-key-in-production"
    ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 1440
    DATABASE_URL: str = "postgresql+asyncpg://postgres:wifi1234@localhost:5432/wifi_db"

    # SINGLE TENANT LOCK this locks the system from being  a saas to single product this i converted
 tp product this harcoded 111 simplify the
    LOCAL_TENANT_ID: str = "11111111-1111-1111-1111-111111111111"

settings = Settings()
```

backend/app/core/exceptions.py

```
from fastapi import HTTPException, status

class TenantNotFoundError(HTTPException):
    def __init__(self):
        super().__init__(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="The requested ISP/Tenant was not found or is inactive."
        )

class PaymentVerificationError(HTTPException):
    def __init__(self, detail: str = "Payment could not be verified."):
        super().__init__(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=detail
        )

class RouterConnectionError(HTTPException):
    def __init__(self):
        super().__init__(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail="The router is currently offline or unreachable."
        )
```

backend/app/core/security.py

```
from datetime import datetime, timedelta, timezone
import jwt
from passlib.context import CryptContext
from typing import Optional
```

```
# In production, load this from environment variables (.env)
SECRET_KEY = "your-super-secret-cryptographic-key-change-in-production"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 1440  # 24 hours

# Bcrypt setup for secure password hashing
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.now(timezone.utc) + expires_delta
    else:
        expire = datetime.now(timezone.utc) + timedelta(minutes=15)

    to_encode.update({"exp": expire})
    # Sign the token
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt
```

backend/app/db/models/model.py

```
import uuid
from datetime import datetime, timezone
from typing import List, Optional
from sqlalchemy import String, Integer, Numeric, DateTime, ForeignKey, UniqueConstraint, Index
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

def get_utc_now():
    return datetime.now(timezone.utc)
    __tablename__ = "tenants"

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    domain: Mapped[Optional[str]] = mapped_column(String(255), unique=True, index=True)

    # Safaricom Daraja Credentials (Encrypted in production)
    daraja_shortcode: Mapped[Optional[str]] = mapped_column(String(50))
    daraja_consumer_key: Mapped[Optional[str]] = mapped_column(String(255))
    daraja_consumer_secret: Mapped[Optional[str]] = mapped_column(String(255))

    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=get_utc_now)

    # Relationships
    users: Mapped[List["User"]] = relationship(back_populates="tenant", cascade="all, delete-orphan"
    customers: Mapped[List["Customer"]] = relationship(back_populates="tenant", cascade="all, delete
    routers: Mapped[List["Router"]] = relationship(back_populates="tenant", cascade="all, delete-orp
    plans: Mapped[List["Plan"]] = relationship(back_populates="tenant", cascade="all, delete-orphan"


class User(Base):
    """Admin and Reseller accounts."""
    __tablename__ = "users"

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    tenant_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("tenants.id", ondelete="CASCADE"), index
```

```
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True, nullable=False)
    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
    role: Mapped[str] = mapped_column(String(50), default="TENANT_ADMIN") # SUPER_ADMIN, TENANT_ADMI
R

    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=get_utc_now)

    tenant: Mapped["Tenant"] = relationship(back_populates="users")


class Router(Base):
    """MikroTik routers linked to a specific tenant."""
    __tablename__ = "routers"

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    tenant_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("tenants.id", ondelete="CASCADE"), index
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    ip_address: Mapped[str] = mapped_column(String(50), nullable=False)
    api_port: Mapped[int] = mapped_column(Integer, default=8728)
    api_username: Mapped[str] = mapped_column(String(100), nullable=False)
    api_password: Mapped[str] = mapped_column(String(255), nullable=False)
    status: Mapped[str] = mapped_column(String(50), default="OFFLINE")

    tenant: Mapped["Tenant"] = relationship(back_populates="routers")
    subscriptions: Mapped[List["Subscription"]] = relationship(back_populates="router")


class Plan(Base):
    """Internet packages configured by the ISP."""
    __tablename__ = "plans"

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    tenant_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("tenants.id", ondelete="CASCADE"), index
    name: Mapped[str] = mapped_column(String(100), nullable=False)
    price: Mapped[float] = mapped_column(Numeric(10, 2), nullable=False)
    speed_limit: Mapped[str] = mapped_column(String(50), nullable=False) # e.g., "10M/10M"
    validity_hours: Mapped[int] = mapped_column(Integer, nullable=False)
    mikrotik_profile_name: Mapped[str] = mapped_column(String(100), nullable=False)

    tenant: Mapped["Tenant"] = relationship(back_populates="plans")


class Customer(Base):
    """The end-user connecting to the Wi-Fi."""
    __tablename__ = "customers"
    __table_args__ = (
        # Prevent duplicate phone numbers WITHIN the same ISP,
        # but allow two different ISPs to have the same customer.
        UniqueConstraint("tenant_id", "phone_number", name="uq_tenant_phone"),
        # Composite index for rapid captive portal lookups
        Index("ix_tenant_phone", "tenant_id", "phone_number"),
    )

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    tenant_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("tenants.id", ondelete="CASCADE"), index
    phone_number: Mapped[str] = mapped_column(String(15), nullable=False)
    mac_address: Mapped[Optional[str]] = mapped_column(String(17))
    status: Mapped[str] = mapped_column(String(50), default="ACTIVE")

    tenant: Mapped["Tenant"] = relationship(back_populates="customers")
    subscriptions: Mapped[List["Subscription"]] = relationship(back_populates="customer", cascade="a
-orphan")


class Subscription(Base):
    """Active or expired internet sessions."""
    __tablename__ = "subscriptions"
```

```
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    customer_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("customers.id", ondelete="CASCADE"), i
    plan_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("plans.id"), index=True)
    router_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("routers.id"), index=True)

    starts_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=get_utc_now)
    expires_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    status: Mapped[str] = mapped_column(String(50), default="ACTIVE") # ACTIVE, EXPIRED, SUSPENDED

    customer: Mapped["Customer"] = relationship(back_populates="subscriptions")
    plan: Mapped["Plan"] = relationship()
    router: Mapped["Router"] = relationship(back_populates="subscriptions")


class Transaction(Base):
    """M-Pesa payment logs."""
    __tablename__ = "transactions"

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    tenant_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("tenants.id", ondelete="CASCADE"), index
    customer_id: Mapped[Optional[uuid.UUID]] = mapped_column(ForeignKey("customers.id"), index=True)

    amount: Mapped[float] = mapped_column(Numeric(10, 2), nullable=False)
    checkout_request_id: Mapped[str] = mapped_column(String(100), unique=True, index=True) # STK Pus
    mpesa_receipt: Mapped[Optional[str]] = mapped_column(String(50), unique=True)
    status: Mapped[str] = mapped_column(String(50), default="PENDING") # PENDING, SUCCESS, FAILED

    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=get_utc_now)qqq
```

backend/app/db/models.py

```
import uuid
from datetime import datetime, timezone
from typing import List, Optional

from sqlalchemy import String, Integer, Numeric, DateTime, ForeignKey, UniqueConstraint, Index, Bool
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

def get_utc_now():
    return datetime.now(timezone.utc)

class Tenant(Base):
    __tablename__ = "tenants"

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    domain: Mapped[Optional[str]] = mapped_column(String(255), unique=True, index=True)
    daraja_shortcode: Mapped[Optional[str]] = mapped_column(String(50))
    daraja_consumer_key: Mapped[Optional[str]] = mapped_column(String(255))
    daraja_consumer_secret: Mapped[Optional[str]] = mapped_column(String(255))
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=get_utc_now)

    users: Mapped[List["User"]] = relationship(back_populates="tenant", cascade="all, delete-orphan"
    customers: Mapped[List["Customer"]] = relationship(back_populates="tenant", cascade="all, delete
    routers: Mapped[List["Router"]] = relationship(back_populates="tenant", cascade="all, delete-orp
    plans: Mapped[List["Plan"]] = relationship(back_populates="tenant", cascade="all, delete-orphan"

class User(Base):
    __tablename__ = "users"

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    tenant_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("tenants.id", ondelete="CASCADE"), index
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True, nullable=False)
```

```
    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
    role: Mapped[str] = mapped_column(String(50), default="TENANT_ADMIN")
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=get_utc_now)

    tenant: Mapped["Tenant"] = relationship(back_populates="users")

class Router(Base):
    __tablename__ = "routers"

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    tenant_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("tenants.id", ondelete="CASCADE"), index
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    ip_address: Mapped[str] = mapped_column(String(50), nullable=False)
    api_port: Mapped[int] = mapped_column(Integer, default=8728)
    api_username: Mapped[str] = mapped_column(String(100), nullable=False)
    api_password: Mapped[str] = mapped_column(String(255), nullable=False)
    status: Mapped[str] = mapped_column(String(50), default="OFFLINE")

    tenant: Mapped["Tenant"] = relationship(back_populates="routers")
    subscriptions: Mapped[List["Subscription"]] = relationship(back_populates="router")

class Plan(Base):
    __tablename__ = "plans"

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    tenant_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("tenants.id", ondelete="CASCADE"), index
    name: Mapped[str] = mapped_column(String(100), nullable=False)
    price: Mapped[float] = mapped_column(Numeric(10, 2), nullable=False)
    speed_limit: Mapped[str] = mapped_column(String(50), nullable=False)
    validity_hours: Mapped[int] = mapped_column(Integer, nullable=False)
    mikrotik_profile_name: Mapped[str] = mapped_column(String(100), nullable=False, default="default

    tenant: Mapped["Tenant"] = relationship(back_populates="plans")

class Customer(Base):
    __tablename__ = "customers"
    __table_args__ = (
        UniqueConstraint("tenant_id", "phone_number", name="uq_tenant_phone"),
        Index("ix_tenant_phone", "tenant_id", "phone_number"),
    )

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    tenant_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("tenants.id", ondelete="CASCADE"), index
    phone_number: Mapped[str] = mapped_column(String(15), nullable=False)
    mac_address: Mapped[Optional[str]] = mapped_column(String(17))
    status: Mapped[str] = mapped_column(String(50), default="ACTIVE")

    tenant: Mapped["Tenant"] = relationship(back_populates="customers")
    subscriptions: Mapped[List["Subscription"]] = relationship(back_populates="customer", cascade="a
-orphan")

class Subscription(Base):
    __tablename__ = "subscriptions"

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    customer_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("customers.id", ondelete="CASCADE"), i
    plan_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("plans.id"), index=True)
    router_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("routers.id"), index=True)

    starts_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=get_utc_now)
    expires_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    status: Mapped[str] = mapped_column(String(50), default="ACTIVE")

    customer: Mapped["Customer"] = relationship(back_populates="subscriptions")
    plan: Mapped["Plan"] = relationship()
    router: Mapped["Router"] = relationship(back_populates="subscriptions")

class Transaction(Base):
```

```
    __tablename__ = "transactions"

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    tenant_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("tenants.id", ondelete="CASCADE"), index
    customer_id: Mapped[Optional[uuid.UUID]] = mapped_column(ForeignKey("customers.id"), index=True)

    #  Link the transaction to the plan being purchased
    plan_id: Mapped[Optional[uuid.UUID]] = mapped_column(ForeignKey("plans.id"), index=True)

    amount: Mapped[float] = mapped_column(Numeric(10, 2), nullable=False)
    checkout_request_id: Mapped[str] = mapped_column(String(100), unique=True, index=True)
    mpesa_receipt: Mapped[Optional[str]] = mapped_column(String(50), unique=True)
    status: Mapped[str] = mapped_column(String(50), default="PENDING")

    #  Zero-Cost Device Transfer Columns
    transfer_pin: Mapped[Optional[str]] = mapped_column(String(6), unique=True, index=True)
    authorized_mac: Mapped[Optional[str]] = mapped_column(String(17))
    is_transferred: Mapped[bool] = mapped_column(Boolean, default=False)

    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=get_utc_now)

    # Relationships mapped for ease of querying later
    tenant: Mapped["Tenant"] = relationship()
    customer: Mapped[Optional["Customer"]] = relationship()
    plan: Mapped[Optional["Plan"]] = relationship()
```

backend/app/db/session.py

```
from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine, async_sessionmaker
from app.core.config import settings

# Create the async database engine with a connection pool
engine = create_async_engine(
    settings.DATABASE_URL,
    echo=False,          # Set to True only for debugging raw SQL queries
    pool_size=20,        # Maximum number of permanent connections
    max_overflow=10,     # How many extra connections to allow during traffic spikes
    pool_recycle=1800    # Reconnect after 30 minutes to prevent dropped connections
)

# This Create a highly reusable session factory
async_session_maker = async_sessionmaker(
    bind=engine,
    class_=AsyncSession,
    expire_on_commit=False,
    autoflush=False
)

# Dependency to inject the database session into FastAPI routes
async def get_db():
    """Yields a database session and safely closes it after the request."""
    async with async_session_maker() as session:
        try:
            yield session
        except Exception:
            await session.rollback()
            raise
        finally:
            await session.close()
```

backend/app/main.py

```
# app/main.py
from fastapi import FastAPI
```

```
from fastapi.middleware.cors import CORSMiddleware

#
from app.api.v1.endpoints import auth, mpesa, admin

app = FastAPI(
    title="Wi-Fi Billing SaaS API",
    description="Multi-tenant backend for ISP hotspot management.",
    version="1.0.0"
)

# Configure CORS for the React Frontend
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "https://production-frontend.com"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Register API Routers
app.include_router(auth.router, prefix="/api/v1/auth", tags=["Authentication"])
app.include_router(mpesa.router, prefix="/api/v1/mpesa", tags=["M-Pesa Integration"])
app.include_router(admin.router, prefix="/api/v1/admin", tags=["Admin Dashboard"])
# app.include_router(routers.router, prefix="/api/v1/routers", tags=["Router Management"])

@app.get("/health")
async def health_check():
    return {"status": "online", "message": "Wi-Fi SaaS Backend is running."}
```

backend/app/services/billing_service.py

```
import uuid
from datetime import datetime, timedelta, timezone
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select

from app.db.models import Subscription, Plan, Customer

async def create_or_extend_subscription(
    db: AsyncSession,
    customer_id: uuid.UUID,
    plan_id: uuid.UUID,
    router_id: uuid.UUID
) -> Subscription:
    """
    Creates a new subscription or extends an existing active one.
    """
    now = datetime.now(timezone.utc)

    # Fetch the plan to get validity hours
    stmt_plan = select(Plan).where(Plan.id == plan_id)
    plan = (await db.execute(stmt_plan)).scalar_one_or_none()

    if not plan:
        raise ValueError("Invalid Plan ID")

    # Checks for exxsting active subscribtion
    stmt_sub = select(Subscription).where(
        Subscription.customer_id == customer_id,
        Subscription.status == 'ACTIVE'
    )
    existing_sub = (await db.execute(stmt_sub)).scalar_one_or_none()

    if existing_sub:
        # Extend the current expiration date
```

```
        # If expired slightly, start from 'now', otherwise add to existing expiry
        base_time = existing_sub.expires_at if existing_sub.expires_at > now else now
        existing_sub.expires_at = base_time + timedelta(hours=plan.validity_hours)
        existing_sub.plan_id = plan_id # Update plan if they bought a different speed
        return existing_sub
    else:
        # Create a brand new subscription
        new_sub = Subscription(
            customer_id=customer_id,
            plan_id=plan_id,
            router_id=router_id,
            starts_at=now,
            expires_at=now + timedelta(hours=plan.validity_hours),
            status='ACTIVE'
        )
        db.add(new_sub)
        return new_sub
```

backend/app/services/mikrotik_service.py

```
import asyncio
import routeros_api
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
import uuid

from app.db.models import Router

# SYNCHRONOUS CORE OPERATIONS

def _connect_to_router(host: str, username: str, password: str, port: int = 8728):
    """Establishes the synchronous socket connection to the router."""
    # NIn production, switch plaintext_login to False and use API-SSL (port 8729)
    connection = routeros_api.RouterOsApiPool(
        host=host,
        username=username,
        password=password,
        port=port,
        plaintext_login=True
    )
    return connection

def _sync_activate_hotspot_user(host, user, pwd, port, customer_phone, plan_profile):
    """Creates or un-suspends a hotspot user and assigns their bandwidth profile."""
    connection = _connect_to_router(host, user, pwd, port)
    api = connection.get_api()

    try:
        hotspot_users = api.get_resource('/ip/hotspot/user')

        # Check if user already exists (Idempotency)
        existing_user = hotspot_users.get(name=customer_phone)

        if existing_user:
            # User exists: Un-disable them and update their profile/speed limit
            user_id = existing_user[0]['id']
            hotspot_users.set(id=user_id, disabled='no', profile=plan_profile)
        else:
            # New User: Create them. We use phone number for both name and password
            hotspot_users.add(
                server='all',
                name=customer_phone,
                password=customer_phone,
                profile=plan_profile
            )
        return True
```

```
    except Exception as e:
        print(f"RouterOS API Error (Activation): {str(e)}")
        raise
    finally:
        connection.disconnect()

def _sync_suspend_hotspot_user(host, user, pwd, port, customer_phone):
    """Disables the hotspot user and forcibly drops their active internet session."""
    connection = _connect_to_router(host, user, pwd, port)
    api = connection.get_api()

    try:
        # 1. Disable the account so they cannot log back in
        hotspot_users = api.get_resource('/ip/hotspot/user')
        existing_user = hotspot_users.get(name=customer_phone)

        if existing_user:
            hotspot_users.set(id=existing_user[0]['id'], disabled='yes')

        # 2. Kick them off immediately (Kill active session)
        active_sessions = api.get_resource('/ip/hotspot/active')
        active_user = active_sessions.get(user=customer_phone)

        for session in active_user:
            active_sessions.remove(id=session['id'])

        return True
    except Exception as e:
        print(f"RouterOS API Error (Suspension): {str(e)}")
        raise
    finally:
        connection.disconnect()

def _sync_get_router_health(host, user, pwd, port):
    """Fetches CPU load, uptime, and active user count for the Admin Dashboard."""
    connection = _connect_to_router(host, user, pwd, port)
    api = connection.get_api()

    try:
        # Get system resources
        resources = api.get_resource('/system/resource').get()[0]
        # Get active user count
        active_users = api.get_resource('/ip/hotspot/active').get()

        return {
            "cpu_load": resources.get('cpu-load', '0') + "%",
            "uptime": resources.get('uptime', '0s'),
            "free_memory": int(resources.get('free-memory', 0)),
            "active_hotspot_users": len(active_users)
        }
    finally:
        connection.disconnect()


# ASYNCHRONOUS FASTAPI WRAPPERS

async def get_router_credentials(db: AsyncSession, tenant_id: str):
    """Helper to fetch the active router for a tenant."""
    # For a multi-router setup, you would pass router_id.
    # Assuming 1 primary router per tenant for this logic:
    stmt = select(Router).where(Router.tenant_id == uuid.UUID(tenant_id), Router.status == 'ONLINE')
    result = await db.execute(stmt)
    router = result.scalars().first()

    if not router:
        raise ValueError(f"No online router found for tenant {tenant_id}")

    return router
```

```
async def activate_customer_on_router(tenant_id: str, phone: str, plan_profile: str, db: AsyncSessio
    """
    Called by the M-Pesa background task after a successful payment.
    Non-blocking wrapper around the RouterOS activation sequence.
    """
    router = await get_router_credentials(db, tenant_id)

    await asyncio.to_thread(
        _sync_activate_hotspot_user,
        host=router.ip_address,
        user=router.api_username,
        pwd=router.api_password,
        port=router.api_port,
        customer_phone=phone,
        plan_profile=plan_profile
    )

async def suspend_customer_on_router(tenant_id: str, phone: str, db: AsyncSession):
    """
    Called by the billing expiration scheduler.
    Non-blocking wrapper around the RouterOS suspension sequence.
    """
    router = await get_router_credentials(db, tenant_id)

    await asyncio.to_thread(
        _sync_suspend_hotspot_user,
        host=router.ip_address,
        user=router.api_username,
        pwd=router.api_password,
        port=router.api_port,
        customer_phone=phone# this will be the customer registered number
    )
```

backend/app/services/mpesa_service.py

```
import base64
from datetime import datetime, timezone
import httpx
from typing import Optional, Tuple

class DarajaService:
    def __init__(self, consumer_key: str, consumer_secret: str, shortcode: str, passkey: str, env: s
box"):
        self.consumer_key = consumer_key
        self.consumer_secret = consumer_secret
        self.shortcode = shortcode
        self.passkey = passkey

        # Base URLs for Safaricom Daraja
        if env == "production":
            self.base_url = "https://api.safaricom.co.kr/mpesa" # Production URL
        else:
            self.base_url = "https://sandbox.safaricom.co.ke"  # Sandbox URL

        # Internal token cache memory to respect rate limits
        self._access_token: Optional[str] = None
        self._token_expires_at: Optional[float] = None

    async def _get_oauth_token(self) -> str:
        """
        Generates and returns a valid Safaricom OAuth Access Token.
        Utilizes internal caching to minimize external API calls.
        """
        import time
        # Check if cached token is still valid (with a 60-second safety buffer)
```

```
        if self._access_token and self._token_expires_at and time.time() < (self._token_expires_at -
            return self._access_token

        # Generate basic auth string
        keys_string = f"{self.consumer_key}:{self.consumer_secret}"
        encoded_keys = base64.b64encode(keys_string.encode("utf-8")).decode("utf-8")

        headers = {"Authorization": f"Basic {encoded_keys}"}
        url = f"{self.base_url}/oauth/v1/generate?grant_type=client_credentials"

        async with httpx.AsyncClient() as client:
            try:
                response = await client.get(url, headers=headers, timeout=10.0)
                response.raise_for_status()
                data = response.json()

                # Cache the new token details
                self._access_token = data["access_token"]
                # Safaricom tokens typically last 3600 seconds (1 hour)
                self._token_expires_at = time.time() + int(data["expires_in"])

                return self._access_token
            except httpx.HTTPStatusError as e:
                raise RuntimeError(f"Daraja OAuth Authentication Failed: {e.response.text}")
            except Exception as e:
                raise RuntimeError(f"Network error during Daraja OAuth: {str(e)}")

    def _generate_timestamp_and_password(self) -> Tuple[str, str]:
        """
        Generates the specialized cryptographic timestamp and password string
        required for Lipa Na M-Pesa Online (STK Push).
        Format: Base64(Shortcode + Passkey + Timestamp)
        """
        # Format must be exactly YYYYMMDDHHmmss in East Africa Time (or UTC equivalent offset)
        # Daraja strictly expects 14 characters
        timestamp = datetime.now(timezone.utc).strftime("%Y%m%d%H%M%S")

        raw_password = f"{self.shortcode}{self.passkey}{timestamp}"
        password = base64.b64encode(raw_password.encode("utf-8")).decode("utf-8")

        return timestamp, password

    async def initiate_stk_push(
        self,
        phone_number: str,
        amount: int,
        callback_url: str,
        account_reference: str,
        transaction_desc: str
    ) -> dict:
        """
        Triggers an M-Pesa STK Push request to a customer's phone handset.

        :param phone_number: Format must be 2547XXXXXXXX or 2541XXXXXXXX
        :param amount: Whole integer value in KSh
        :param callback_url: Secure HTTPS API endpoint hosted on your FastAPI server
        :param account_reference: Visible label to user (e.g., WiFi Account Number)
        :param transaction_desc: Internal text describing intent
        """
        token = await self._get_oauth_token()
        timestamp, password = self._generate_timestamp_and_password()

        headers = {
            "Authorization": f"Bearer {token}",
            "Content-Type": "application/json"
        }

        payload = {
```

```
            "BusinessShortCode": self.shortcode,
            "Password": password,
            "Timestamp": timestamp,
            "TransactionType": "CustomerPayBillOnline", # CustomerBuyGoodsOnline for Till Numbers
            "Amount": int(amount),
            "PartyA": phone_number,
            "PartyB": self.shortcode,
            "PhoneNumber": phone_number,
            "CallBackURL": callback_url,
            "AccountReference": account_reference,
            "TransactionDesc": transaction_desc
        }

        url = f"{self.base_url}/mpesa/stkpush/v1/query" if False else f"{self.base_url}/mpesa/stkpus
ssrequest"

        async with httpx.AsyncClient() as client:
            try:
                response = await client.post(url, json=payload, headers=headers, timeout=15.0)
                response_data = response.json()

                # Check for successful handshake from Safaricom (ResponseCode '0' means request acce
                if response_data.get("ResponseCode") == "0":
                    return {
                        "success": True,
                        "merchant_request_id": response_data.get("MerchantRequestID"),
                        "checkout_request_id": response_data.get("CheckoutRequestID"),
                        "description": response_data.get("ResponseDescription")
                    }
                else:
                    return {
                        "success": False,
                        "error": response_data.get("ResponseDescription", "Unknown Daraja rejection
                    }
            except httpx.HTTPStatusError as e:
                return {"success": False, "error": f"Safaricom Server Error: {e.response.text}"}
            except Exception as e:
                return {"success": False, "error": f"Connection Failure: {str(e)}"}
```

backend/app/services/sms_service.py

```
import africastalking

# Initialize Africa's Talking // sandbox details
AT_USERNAME = "your_at_username"
AT_API_KEY = "your_at_api_key"

africastalking.initialize(AT_USERNAME, AT_API_KEY)
sms = africastalking.SMS

def send_sms_alert(phone_number: str, message: str, sender_id: str = None):
    """
    Sends an SMS using Africa's Talking.
    Phone numbers must be in E.164 format (e.g., +254712345678).
    """
    try:
        # If the phone number lacks the country code, format it for Kenya
        if phone_number.startswith("0"):
            phone_number = "+254" + phone_number[1:]
        elif phone_number.startswith("254"):
            phone_number = "+" + phone_number

        kwargs = {
            "to": [phone_number],
            "message": message
        }
```

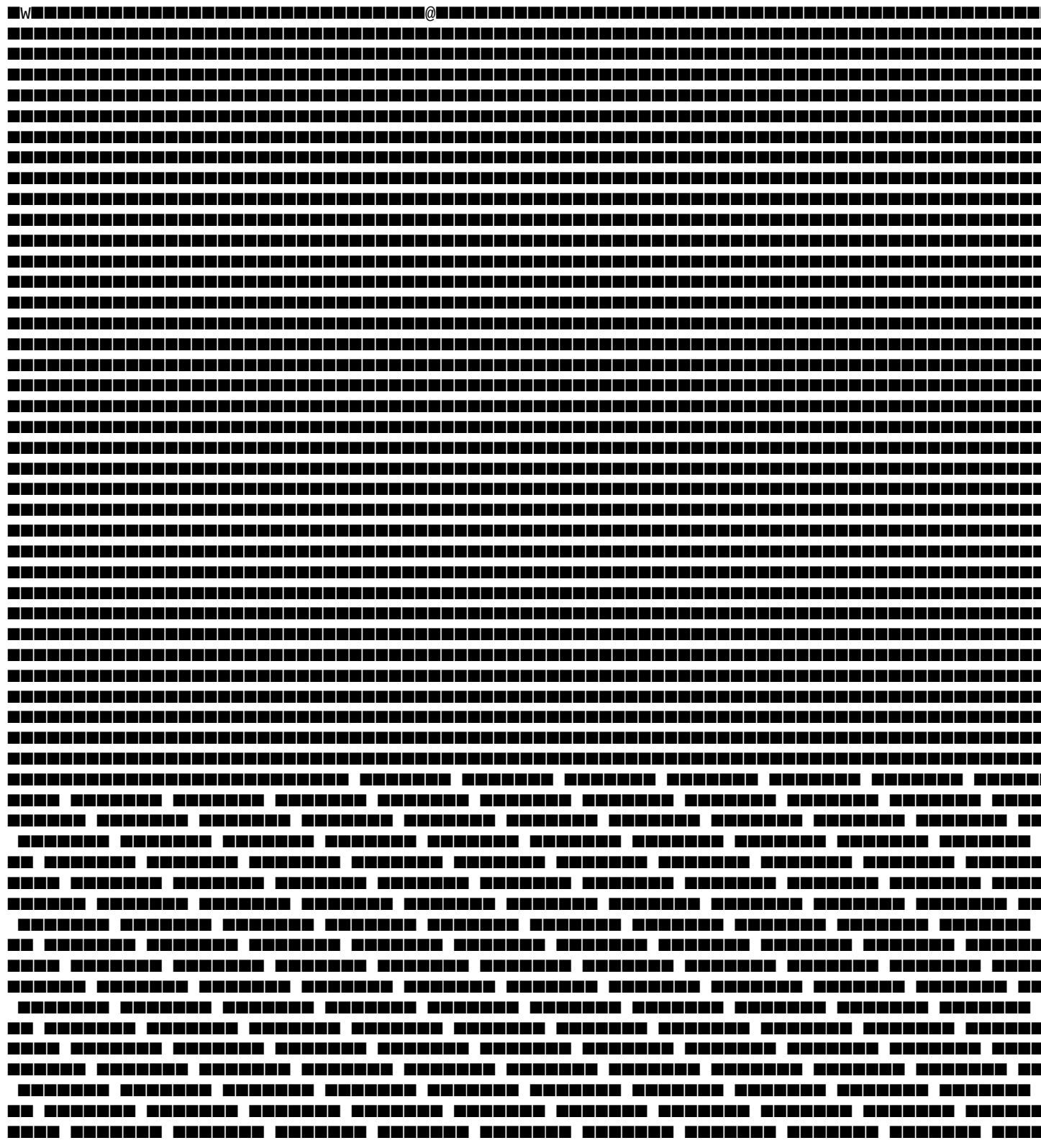
```
        if sender_id:
            kwargs["from_"] = sender_id # e.g., "MOBILINK"

        response = sms.send(**kwargs)
        return response
    except Exception as e:
        print(f"Failed to send SMS to {phone_number}: {str(e)}")
        return None
```

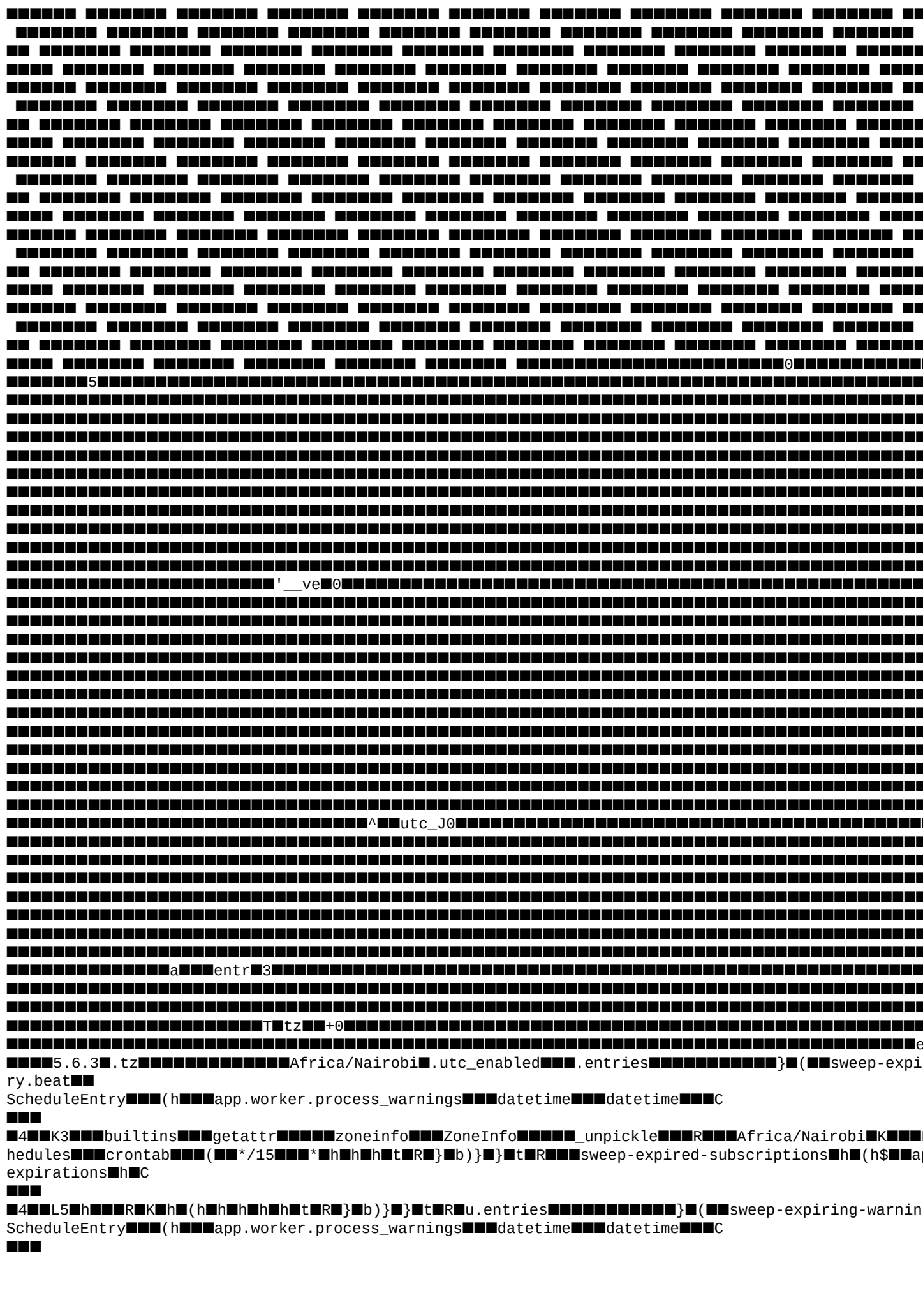
backend/app/worker.py

```
import asyncio
from datetime import datetime, timezone
from celery import Celery
from celery.schedules import crontab
from sqlalchemy import select
from sqlalchemy.orm import selectinload
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker
from sqlalchemy.pool import NullPool

from app.core.config import settings
from app.db.models import Subscription, Customer

# Initialize Celery pointing to local Redis
celery_app = Celery(
    "wifi_billing_worker",
    broker="redis://localhost:6379/0",
    backend="redis://localhost:6379/0"
)

celery_app.conf.beat_schedule = {
    'sweep-expired-subscriptions': {
        'task': 'app.worker.process_expirations',
        'schedule': crontab(minute='*'),
    }
}
celery_app.conf.timezone = 'Africa/Nairobi'

async def _async_process_expirations():
    print("Sweeping database for expired subscriptions...")
    now = datetime.now(timezone.utc)

    # Create a fresh engine inside the async loop with NullPool
    # This prevents the "attached to a different loop" RuntimeError
    engine = create_async_engine(settings.DATABASE_URL, poolclass=NullPool)
    local_session = async_sessionmaker(engine, expire_on_commit=False)

    try:
        async with local_session() as db:
            stmt = select(Subscription).options(
                selectinload(Subscription.customer).selectinload(Customer.tenant)
            ).where(
                Subscription.status == 'ACTIVE',
                Subscription.expires_at <= now
            )
            result = await db.execute(stmt)
            expired_subs = result.scalars().all()

            for sub in expired_subs:
                print(f"Disconnecting customer {sub.customer.phone_number} on router {sub.router_id}")
                sub.status = 'EXPIRED'

            if expired_subs:
                await db.commit()
                print(f"Successfully expired {len(expired_subs)} accounts.")
            else:
```

```
                print("No expired accounts found this minute.")
    finally:
        # Safely tear down the connections linked to this specific event loop
        await engine.dispose()

@celery_app.task
def process_expirations():
    asyncio.run(_async_process_expirations())
```

backend/celerybeat-schedule

```

```



■4■■K3■■■builtins■■■getattr■■■■■zoneinfo■■■ZoneInfo■■■■■_unpickle■■■R■■■Africa/Nairobi■K■■■
hedules■■■crontab■■■(■■*/15■■■*■h■h■h■t■R■}■b)}■}■t■R■■■sweep-expired-subscriptions■h■(■■swe
ptions■■■app.worker.process_expirations■h■C
■■■
■8■■■■■h■■■R■K■h■(h■h■h■h■h■t■R■}■b)}■}■t■R■u.entries■■■■■■■■■■■}■(■■sweep-expiring-warnin
ScheduleEntry■■■(■■sweep-expiring-warnings■■■app.worker.process_warnings■■■datetime■■■datetime■■
■■■
■■■■■■■■builtins■■■getattr■■■■■zoneinfo■■■ZoneInfo■■■■■_unpickle■■■R■■■Africa/Nairobi■K■■■R
edules■■■crontab■■■(■■*/15■■■*■h■h■h■t■R■}■b)}■}■t■R■■■sweep-expired-subscriptions■h■(■■sweep
tions■■■app.worker.process_expirations■h■C
■■■
■)■■■■h■■■R■K■h■(h■h■h■h■h■t■R■}■b)}■}■t■R■u.■■■
■■
■■
■■
■■
■■
■■
■■
■■
■■
■■
■■
■■■
■■■
■■■
■■■
■■■
■■■
■■■
■■■
■■■
■■■
■■■
■■■
■■■
■■■
■■■
■■■
■■
```

### backend/docker-compose.yml

```

```

### backend/Dockerfile

```

```

### backend/migrations/env.py

```
import asyncio
from logging.config import fileConfig

from sqlalchemy import pool
from sqlalchemy.engine import Connection
from sqlalchemy.ext.asyncio import async_engine_from_config

from alembic import context

# this is the Alembic Config object, which provides
# access to the values within the .ini file in use.
config = context.config

Interpret the config file for Python logging.
This line sets up loggers basically.
```
```

```
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

# add your model's MetaData object here
# for 'autogenerate' support
# from myapp import mymodel
# target_metadata = mymodel.Base.metadata
from app.db.models import Base
from app.core.config import settings

target_metadata = Base.metadata
config.set_main_option("sqlalchemy.url", settings.DATABASE_URL)
# other values from the config, defined by the needs of env.py,
# can be acquired:
# my_important_option = config.get_main_option("my_important_option")
# ... etc.


def run_migrations_offline() -> None:
    """Run migrations in 'offline' mode.

    This configures the context with just a URL
    and not an Engine, though an Engine is acceptable
    here as well.  By skipping the Engine creation
    we don't even need a DBAPI to be available.

    Calls to context.execute() here emit the given string to the
    script output.

    """
    url = config.get_main_option("sqlalchemy.url")
    context.configure(
        url=url,
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )

    with context.begin_transaction():
        context.run_migrations()


def do_run_migrations(connection: Connection) -> None:
    context.configure(connection=connection, target_metadata=target_metadata)

    with context.begin_transaction():
        context.run_migrations()


async def run_async_migrations() -> None:
    """In this scenario we need to create an Engine
    and associate a connection with the context.

    """

    connectable = async_engine_from_config(
        config.get_section(config.config_ini_section, {}),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )

    async with connectable.connect() as connection:
        await connection.run_sync(do_run_migrations)

    await connectable.dispose()


def run_migrations_online() -> None:
```

```
    """Run migrations in 'online' mode."""

    asyncio.run(run_async_migrations())


if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()
```

backend/migrations/README

```
Generic single-database configuration with an async dbapi.
```

backend/migrations/script.py.mako

```
"""${message}

Revision ID: ${up_revision}
Revises: ${down_revision | comma,n}
Create Date: ${create_date}

"""
from typing import Sequence, Union

from alembic import op
import sqlalchemy as sa
${imports if imports else ""}

# revision identifiers, used by Alembic.
revision: str = ${repr(up_revision)}
down_revision: Union[str, Sequence[str], None] = ${repr(down_revision)}
branch_labels: Union[str, Sequence[str], None] = ${repr(branch_labels)}
depends_on: Union[str, Sequence[str], None] = ${repr(depends_on)}


def upgrade() -> None:
    """Upgrade schema."""
    ${upgrades if upgrades else "pass"}


def downgrade() -> None:
    """Downgrade schema."""
    ${downgrades if downgrades else "pass"}
```

backend/migrations/versions/b9b0bb9c2949_initial_saas_tables.py

```
"""Initial SaaS tables

Revision ID: b9b0bb9c2949
Revises:
Create Date: 2026-06-13 11:40:22.858262

"""
from typing import Sequence, Union

from alembic import op
import sqlalchemy as sa
```

```
# revision identifiers, used by Alembic.
revision: str = 'b9b0bb9c2949'
down_revision: Union[str, Sequence[str], None] = None
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None


def upgrade() -> None:
    """Upgrade schema."""
    # ### commands auto generated by Alembic - please adjust! ###
    op.create_table('tenants',
    sa.Column('id', sa.Uuid(), nullable=False),
    sa.Column('name', sa.String(length=255), nullable=False),
    sa.Column('domain', sa.String(length=255), nullable=True),
    sa.Column('daraja_shortcode', sa.String(length=50), nullable=True),
    sa.Column('daraja_consumer_key', sa.String(length=255), nullable=True),
    sa.Column('daraja_consumer_secret', sa.String(length=255), nullable=True),
    sa.Column('created_at', sa.DateTime(timezone=True), nullable=False),
    sa.PrimaryKeyConstraint('id')
    )
    op.create_index(op.f('ix_tenants_domain'), 'tenants', ['domain'], unique=True)
    op.create_table('customers',
    sa.Column('id', sa.Uuid(), nullable=False),
    sa.Column('tenant_id', sa.Uuid(), nullable=False),
    sa.Column('phone_number', sa.String(length=15), nullable=False),
    sa.Column('mac_address', sa.String(length=17), nullable=True),
    sa.Column('status', sa.String(length=50), nullable=False),
    sa.ForeignKeyConstraint(['tenant_id'], ['tenants.id'], ondelete='CASCADE'),
    sa.PrimaryKeyConstraint('id'),
    sa.UniqueConstraint('tenant_id', 'phone_number', name='uq_tenant_phone')
    )
    op.create_index(op.f('ix_customers_tenant_id'), 'customers', ['tenant_id'], unique=False)
    op.create_index('ix_tenant_phone', 'customers', ['tenant_id', 'phone_number'], unique=False)
    op.create_table('plans',
    sa.Column('id', sa.Uuid(), nullable=False),
    sa.Column('tenant_id', sa.Uuid(), nullable=False),
    sa.Column('name', sa.String(length=100), nullable=False),
    sa.Column('price', sa.Numeric(precision=10, scale=2), nullable=False),
    sa.Column('speed_limit', sa.String(length=50), nullable=False),
    sa.Column('validity_hours', sa.Integer(), nullable=False),
    sa.Column('mikrotik_profile_name', sa.String(length=100), nullable=False),
    sa.ForeignKeyConstraint(['tenant_id'], ['tenants.id'], ondelete='CASCADE'),
    sa.PrimaryKeyConstraint('id')
    )
    op.create_index(op.f('ix_plans_tenant_id'), 'plans', ['tenant_id'], unique=False)
    op.create_table('routers',
    sa.Column('id', sa.Uuid(), nullable=False),
    sa.Column('tenant_id', sa.Uuid(), nullable=False),
    sa.Column('name', sa.String(length=255), nullable=False),
    sa.Column('ip_address', sa.String(length=50), nullable=False),
    sa.Column('api_port', sa.Integer(), nullable=False),
    sa.Column('api_username', sa.String(length=100), nullable=False),
    sa.Column('api_password', sa.String(length=255), nullable=False),
    sa.Column('status', sa.String(length=50), nullable=False),
    sa.ForeignKeyConstraint(['tenant_id'], ['tenants.id'], ondelete='CASCADE'),
    sa.PrimaryKeyConstraint('id')
    )
    op.create_index(op.f('ix_routers_tenant_id'), 'routers', ['tenant_id'], unique=False)
    op.create_table('users',
    sa.Column('id', sa.Uuid(), nullable=False),
    sa.Column('tenant_id', sa.Uuid(), nullable=False),
    sa.Column('email', sa.String(length=255), nullable=False),
    sa.Column('password_hash', sa.String(length=255), nullable=False),
    sa.Column('role', sa.String(length=50), nullable=False),
    sa.Column('created_at', sa.DateTime(timezone=True), nullable=False),
    sa.ForeignKeyConstraint(['tenant_id'], ['tenants.id'], ondelete='CASCADE'),
    sa.PrimaryKeyConstraint('id')
    )
```

```
    op.create_index(op.f('ix_users_email'), 'users', ['email'], unique=True)
    op.create_index(op.f('ix_users_tenant_id'), 'users', ['tenant_id'], unique=False)
    op.create_table('subscriptions',
    sa.Column('id', sa.Uuid(), nullable=False),
    sa.Column('customer_id', sa.Uuid(), nullable=False),
    sa.Column('plan_id', sa.Uuid(), nullable=False),
    sa.Column('router_id', sa.Uuid(), nullable=False),
    sa.Column('starts_at', sa.DateTime(timezone=True), nullable=False),
    sa.Column('expires_at', sa.DateTime(timezone=True), nullable=False),
    sa.Column('status', sa.String(length=50), nullable=False),
    sa.ForeignKeyConstraint(['customer_id'], ['customers.id'], ondelete='CASCADE'),
    sa.ForeignKeyConstraint(['plan_id'], ['plans.id'], ),
    sa.ForeignKeyConstraint(['router_id'], ['routers.id'], ),
    sa.PrimaryKeyConstraint('id')
    )
    op.create_index(op.f('ix_subscriptions_customer_id'), 'subscriptions', ['customer_id'], unique=F
    op.create_index(op.f('ix_subscriptions_plan_id'), 'subscriptions', ['plan_id'], unique=False)
    op.create_index(op.f('ix_subscriptions_router_id'), 'subscriptions', ['router_id'], unique=False
    op.create_table('transactions',
    sa.Column('id', sa.Uuid(), nullable=False),
    sa.Column('tenant_id', sa.Uuid(), nullable=False),
    sa.Column('customer_id', sa.Uuid(), nullable=True),
    sa.Column('amount', sa.Numeric(precision=10, scale=2), nullable=False),
    sa.Column('checkout_request_id', sa.String(length=100), nullable=False),
    sa.Column('mpesa_receipt', sa.String(length=50), nullable=True),
    sa.Column('status', sa.String(length=50), nullable=False),
    sa.Column('created_at', sa.DateTime(timezone=True), nullable=False),
    sa.ForeignKeyConstraint(['customer_id'], ['customers.id'], ),
    sa.ForeignKeyConstraint(['tenant_id'], ['tenants.id'], ondelete='CASCADE'),
    sa.PrimaryKeyConstraint('id'),
    sa.UniqueConstraint('mpesa_receipt')
    )
    op.create_index(op.f('ix_transactions_checkout_request_id'), 'transactions', ['checkout_request_
ue=True)
    op.create_index(op.f('ix_transactions_customer_id'), 'transactions', ['customer_id'], unique=Fal
    op.create_index(op.f('ix_transactions_tenant_id'), 'transactions', ['tenant_id'], unique=False)
    # ### end Alembic commands ###


def downgrade() -> None:
    """Downgrade schema."""
    # ### commands auto generated by Alembic - please adjust! ###
    op.drop_index(op.f('ix_transactions_tenant_id'), table_name='transactions')
    op.drop_index(op.f('ix_transactions_customer_id'), table_name='transactions')
    op.drop_index(op.f('ix_transactions_checkout_request_id'), table_name='transactions')
    op.drop_table('transactions')
    op.drop_index(op.f('ix_subscriptions_router_id'), table_name='subscriptions')
    op.drop_index(op.f('ix_subscriptions_plan_id'), table_name='subscriptions')
    op.drop_index(op.f('ix_subscriptions_customer_id'), table_name='subscriptions')
    op.drop_table('subscriptions')
    op.drop_index(op.f('ix_users_tenant_id'), table_name='users')
    op.drop_index(op.f('ix_users_email'), table_name='users')
    op.drop_table('users')
    op.drop_index(op.f('ix_routers_tenant_id'), table_name='routers')
    op.drop_table('routers')
    op.drop_index(op.f('ix_plans_tenant_id'), table_name='plans')
    op.drop_table('plans')
    op.drop_index('ix_tenant_phone', table_name='customers')
    op.drop_index(op.f('ix_customers_tenant_id'), table_name='customers')
    op.drop_table('customers')
    op.drop_index(op.f('ix_tenants_domain'), table_name='tenants')
    op.drop_table('tenants')
    # ### end Alembic commands ###

```

backend/requirements.txt

```

```

frontend files

frontend/eslint.config.js

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import tseslint from 'typescript-eslint'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{ts,tsx}'],
    extends: [
      js.configs.recommended,
      tseslint.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      globals: globals.browser,
    },
  },
])
```

frontend/.gitignore

```
# Logs
logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
pnpm-debug.log*
lerna-debug.log*

node_modules
dist
dist-ssr
*.local

# Editor directories and files
.vscode/*
!.vscode/extensions.json
.idea
.DS_Store
*.suo
*.ntvs*
*.njsproj
*.sln
*.sw?

```

frontend/index.html

```
<!doctype html>
```

```
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>frontend</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

frontend/package.json

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.6",
    "react-dom": "^19.2.6",
    "react-router-dom": "^7.18.0"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/node": "^24.12.3",
    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.1",
    "eslint": "^10.3.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.6.0",
    "typescript": "~6.0.2",
    "typescript-eslint": "^8.59.2",
    "vite": "^8.0.12"
  }
}
```

frontend/package-lock.json

<!-- Truncated to first 20000 characters -->

```
{
  "name": "frontend",
  "version": "0.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "frontend",
      "version": "0.0.0",
      "dependencies": {
        "react": "^19.2.6",
```

```
        "react-dom": "^19.2.6",
        "react-router-dom": "^7.18.0"
      },
      "devDependencies": {
        "@eslint/js": "^10.0.1",
        "@types/node": "^24.12.3",
        "@types/react": "^19.2.14",
        "@types/react-dom": "^19.2.3",
        "@vitejs/plugin-react": "^6.0.1",
        "eslint": "^10.3.0",
        "eslint-plugin-react-hooks": "^7.1.1",
        "eslint-plugin-react-refresh": "^0.5.2",
        "globals": "^17.6.0",
        "typescript": "~6.0.2",
        "typescript-eslint": "^8.59.2",
        "vite": "^8.0.12"
      }
    },
    "node_modules/@babel/code-frame": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/code-frame/-/code-frame-7.29.7.tgz",
      "integrity": "sha512-Aup7aUOfpbAUg2ROOJN6Iw5f9DMBlzu0mIkm/malLQFN/YQgO48wCj0Kxa3sEHJvPVFg7siR+
QKw==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@babel/helper-validator-identifier": "^7.29.7",
        "js-tokens": "^4.0.0",
        "picocolors": "^1.1.1"
      },
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/compat-data": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/compat-data/-/compat-data-7.29.7.tgz",
      "integrity": "sha512-locTkQyKvwIEgBzVrn8693ebc97F2U8ZHjbXwDXJ5Fn2TCpNwTlKcaKLkdHop5c/icOFE7qt7
6Gg==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/core": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/core/-/core-7.29.7.tgz",
      "integrity": "sha512-RgHBCvtjbOK2gXSNBNIkNoEc9qoVEtau3hj8gEqKQuL3HZAibKarWFEI3Lfm6EYKkLalOh8eS
VBA==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@babel/code-frame": "^7.29.7",
        "@babel/generator": "^7.29.7",
        "@babel/helper-compilation-targets": "^7.29.7",
        "@babel/helper-module-transforms": "^7.29.7",
        "@babel/helpers": "^7.29.7",
        "@babel/parser": "^7.29.7",
        "@babel/template": "^7.29.7",
        "@babel/traverse": "^7.29.7",
        "@babel/types": "^7.29.7",
        "@jridgewell/remapping": "^2.3.5",
        "convert-source-map": "^2.0.0",
        "debug": "^4.1.0",
        "gensync": "^1.0.0-beta.2",
        "json5": "^2.2.3",
        "semver": "^6.3.1"
```

```
      },
      "engines": {
        "node": ">=6.9.0"
      },
      "funding": {
        "type": "opencollective",
        "url": "https://opencollective.com/babel"
      }
    },
    "node_modules/@babel/generator": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/generator/-/generator-7.29.7.tgz",
      "integrity": "sha512-DkXD5OJQaAQIdZ1bt3UZdEnHAn9Imd3IVBdX03UFe+ony9Ojw5pzr9YVKGDY1jt+Gcn/FnGkN
WtQ==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@babel/parser": "^7.29.7",
        "@babel/types": "^7.29.7",
        "@jridgewell/gen-mapping": "^0.3.12",
        "@jridgewell/trace-mapping": "^0.3.28",
        "jsesc": "^3.0.2"
      },
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/helper-compilation-targets": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/helper-compilation-targets/-/helper-compilation
.29.7.tgz",
      "integrity": "sha512-wem6WaBj4NaVYVdNhLPPVacES6ZJ+KBBfSkTMD3YZxbP3rm3Di85tJU5ljaUNhaOynt+Aj0xr
71g==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@babel/compat-data": "^7.29.7",
        "@babel/helper-validator-option": "^7.29.7",
        "browserslist": "^4.24.0",
        "lru-cache": "^5.1.1",
        "semver": "^6.3.1"
      },
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/helper-globals": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/helper-globals/-/helper-globals-7.29.7.tgz",
      "integrity": "sha512-3nQVUAtvkKH9zahfWgw96Jc/uFOmjACE1kQz82E2lqWmHBgjzbNlsC22nuQTfahmWeQtTq5nQ
4zA==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/helper-module-imports": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/helper-module-imports/-/helper-module-imports-7
,
      "integrity": "sha512-ejHwrQQYcm9xnTivShn2IDOlIzInN34AXskvq9QicvCtEzq1Vzclu/tKF8Jq1Cg8JG2GL6/Em
E3g==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@babel/traverse": "^7.29.7",
        "@babel/types": "^7.29.7"
```

```
      },
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/helper-module-transforms": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/helper-module-transforms/-/helper-module-transf
7.tgz",
      "integrity": "sha512-UPUVSyXbOh627KiCIGQSgwWzGeBKLkaJ9PJEdrngIwMSzxLR4jS4+f1f1jb7VzBbg8nFLaYot
TAg==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@babel/helper-module-imports": "^7.29.7",
        "@babel/helper-validator-identifier": "^7.29.7",
        "@babel/traverse": "^7.29.7"
      },
      "engines": {
        "node": ">=6.9.0"
      },
      "peerDependencies": {
        "@babel/core": "^7.0.0"
      }
    },
    "node_modules/@babel/helper-string-parser": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/helper-string-parser/-/helper-string-parser-7.2
      "integrity": "sha512-Pb5ijPrZ89GDH8223L4UP8i6QApWxs04RbPQJTeWDV0/keR2E36MeKnyr6LYmUUvqRRI+Iv87
zYw==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/helper-validator-identifier": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/helper-validator-identifier/-/helper-validator-
-7.29.7.tgz",
      "integrity": "sha512-qehxGkRj55h/ff8EMaJ+cYhyaKlHIxqYDn682wQD7RNp9UujOQsHog2uS0r2vzr4pW+sXf90N
fFg==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/helper-validator-option": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/helper-validator-option/-/helper-validator-opti
tgz",
      "integrity": "sha512-N9ZErrD+yW5geCDtBqnOoxmR8+tNKiGuxKlDpuJxfsqpa2dFcexaziGAE/qoHLiDDreVNMupx
3gw==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/helpers": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/helpers/-/helpers-7.29.7.tgz",
      "integrity": "sha512-1k2lAGRMfHTcwuNYcCNUmaUffmQv8KWMfh2iJUUeRlwlwH4FdNG7mfPI10NPfLHJFThE4Tyr4
uBg==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
```

```
        "@babel/template": "^7.29.7",
        "@babel/types": "^7.29.7"
      },
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/parser": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/parser/-/parser-7.29.7.tgz",
      "integrity": "sha512-hnORnjP/1P/zFEndoeX+n+t1RwWRJiJpM/jO7FW32Kn9r5+sJB2JWOdYo4L6k78j15eCwY3Gm
tNg==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@babel/types": "^7.29.7"
      },
      "bin": {
        "parser": "bin/babel-parser.js"
      },
      "engines": {
        "node": ">=6.0.0"
      }
    },
    "node_modules/@babel/template": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/template/-/template-7.29.7.tgz",
      "integrity": "sha512-puq+Gf35oI24FeN11LkoUQFqv9uwNeWpxXZi/Ji3rRIoKAzKnxRaZ+Gkj0vKS9ZCiTESfng1N
+Gg==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@babel/code-frame": "^7.29.7",
        "@babel/parser": "^7.29.7",
        "@babel/types": "^7.29.7"
      },
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/traverse": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/traverse/-/traverse-7.29.7.tgz",
      "integrity": "sha512-EhlfNQtZ+NK22w5BM61ciuiq1m58ed33Wr1Xan//ZRTy6hgjnwyCffRYwzsGXdASJSUJ1guZI
+zw==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@babel/code-frame": "^7.29.7",
        "@babel/generator": "^7.29.7",
        "@babel/helper-globals": "^7.29.7",
        "@babel/parser": "^7.29.7",
        "@babel/template": "^7.29.7",
        "@babel/types": "^7.29.7",
        "debug": "^4.3.1"
      },
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/types": {
      "version": "7.29.7",
      "resolved": "https://registry.npmjs.org/@babel/types/-/types-7.29.7.tgz",
      "integrity": "sha512-4zBIxpPzowiZpusoFkyGVwakdRJUyuH5PxQ/PrqghfdFWWasvnCdPfQXHrenDai+gyLARulZj
4pA==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
```

```
        "@babel/helper-string-parser": "^7.29.7",
        "@babel/helper-validator-identifier": "^7.29.7"
      },
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@emnapi/core": {
      "version": "1.10.0",
      "resolved": "https://registry.npmjs.org/@emnapi/core/-/core-1.10.0.tgz",
      "integrity": "sha512-yq6OkJ4p82CAfPl0u9mQebQHKPJkY7WrIuk205cTYnYe+k2Z8YBh11FrbRG/H6ihirqcacOgl
eXw==",
      "dev": true,
      "license": "MIT",
      "optional": true,
      "dependencies": {
        "@emnapi/wasi-threads": "1.2.1",
        "tslib": "^2.4.0"
      }
    },
    "node_modules/@emnapi/runtime": {
      "version": "1.10.0",
      "resolved": "https://registry.npmjs.org/@emnapi/runtime/-/runtime-1.10.0.tgz",
      "integrity": "sha512-ewvYlk86xUoGI0zQRNq/mC+16R1QeDlKQy21Ki3oSYXNgLb45GV1P6A0M+/s6nyCuNDqe5Vpa
wFA==",
      "dev": true,
      "license": "MIT",
      "optional": true,
      "dependencies": {
        "tslib": "^2.4.0"
      }
    },
    "node_modules/@emnapi/wasi-threads": {
      "version": "1.2.1",
      "resolved": "https://registry.npmjs.org/@emnapi/wasi-threads/-/wasi-threads-1.2.1.tgz",
      "integrity": "sha512-uTII7OYF+/Mes/MrcIOYp5yOtSMLBWSIoLPpcgwipoiKbli6k322tcoFsxoIIxPDqW01SQGAg
v2w==",
      "dev": true,
      "license": "MIT",
      "optional": true,
      "dependencies": {
        "tslib": "^2.4.0"
      }
    },
    "node_modules/@eslint-community/eslint-utils": {
      "version": "4.9.1",
      "resolved": "https://registry.npmjs.org/@eslint-community/eslint-utils/-/eslint-utils-4.9.1.tg
      "integrity": "sha512-phrYmNiYppR7znFEdqgfWHXR6NCkZEK7hwWDHZUjit/2/U0r6XvkDl0SYnoM51Hq7FhCGdLDT
xsQ==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "eslint-visitor-keys": "^3.4.3"
      },
      "engines": {
        "node": "^12.22.0 || ^14.17.0 || >=16.0.0"
      },
      "funding": {
        "url": "https://opencollective.com/eslint"
      },
      "peerDependencies": {
        "eslint": "^6.0.0 || ^7.0.0 || >=8.0.0"
      }
    },
    "node_modules/@eslint-community/eslint-utils/node_modules/eslint-visitor-keys": {
      "version": "3.4.3",
      "resolved": "https://registry.npmjs.org/eslint-visitor-keys/-/eslint-visitor-keys-3.4.3.tgz",
      "integrity": "sha512-wpc+LXeiyiisxPlEkUzU6svyS1frIO3Mgxj1fdy7Pm8Ygzguax2N3Fa/D/ag1WqbOprdI+uY6
```

```
iag==",
      "dev": true,
      "license": "Apache-2.0",
      "engines": {
        "node": "^12.22.0 || ^14.17.0 || >=16.0.0"
      },
      "funding": {
        "url": "https://opencollective.com/eslint"
      }
    },
    "node_modules/@eslint-community/regexpp": {
      "version": "4.12.2",
      "resolved": "https://registry.npmjs.org/@eslint-community/regexpp/-/regexpp-4.12.2.tgz",
      "integrity": "sha512-EriSTlt5OC9/7SXkRSCAhfSxxoSUgBm33OH+IkwbdpgoqsSsUg7y3uh+IICI/Qg4BBWr3U2i3
4ew==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": "^12.0.0 || ^14.0.0 || >=16.0.0"
      }
    },
    "node_modules/@eslint/config-array": {
      "version": "0.23.5",
      "resolved": "https://registry.npmjs.org/@eslint/config-array/-/config-array-0.23.5.tgz",
      "integrity": "sha512-Y3kKLvC1dvTOT+oGlqNQ1XLqK6D1HU2YXPc52NmAlJZbMMWDzGYXMiPRJ8TYD39muD/OTjlZm
MBA==",
      "dev": true,
      "license": "Apache-2.0",
      "dependencies": {
        "@eslint/object-schema": "^3.0.5",
        "debug": "^4.3.1",
        "minimatch": "^10.2.4"
      },
      "engines": {
        "node": "^20.19.0 || ^22.13.0 || >=24"
      }
    },
    "node_modules/@eslint/config-helpers": {
      "version": "0.6.0",
      "resolved": "https://registry.npmjs.org/@eslint/config-helpers/-/config-helpers-0.6.0.tgz",
      "integrity": "sha512-ii6Bw9jJ2zi2cWA2Z+9/QZ/+3DX6kwaV5Q986D/CdP3Lap3w/pgQZ373FV7byY/i7L4IRH/G4
bpA==",
      "dev": true,
      "license": "Apache-2.0",
      "dependencies": {
        "@eslint/core": "^1.2.1"
      },
      "engines": {
        "node": "^20.19.0 || ^22.13.0 || >=24"
      }
    },
    "node_modules/@eslint/core": {
      "version": "1.2.1",
      "resolved": "https://registry.npmjs.org/@eslint/core/-/core-1.2.1.tgz",
      "integrity": "sha512-MwcE1P+AZ4C6DWlpin/OmOA54mmIZ/+xZuJiQd4SyB29oAJjN30UW9wkKNptW2ctp4cEsvhlL
loQ==",
      "dev": true,
      "license": "Apache-2.0",
      "dependencies": {
        "@types/json-schema": "^7.0.15"
      },
      "engines": {
        "node": "^20.19.0 || ^22.13.0 || >=24"
      }
    },
    "node_modules/@eslint/js": {
      "version": "10.0.1",
      "resolved": "https://registry.npmjs.org/@eslint/js/-/js-10.0.1.tgz",
```

```
      "integrity": "sha512-zeR9k5pd4gxjZ0abRoIaxdc7I3nDktoXZk2qOv9gCNWx3mVwEn32VRhyLaRsDiJjTs0xq/T8m
BcA==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": "^20.19.0 || ^22.13.0 || >=24"
      },
      "funding": {
        "url": "https://eslint.org/donate"
      },
      "peerDependencies": {
        "eslint": "^10.0.0"
      },
      "peerDependenciesMeta": {
        "eslint": {
          "optional": true
        }
      }
    },
    "node_modules/@eslint/object-schema": {
      "version": "3.0.5",
      "resolved": "https://registry.npmjs.org/@eslint/object-schema/-/object-schema-3.0.5.tgz",
      "integrity": "sha512-vqTaUEgxzm+YDSdElad6PiRoX4t8VGDjCtt05zn4nU810UIx/uNEV7/lZJ6KwFThKZOzOxzXy
aMw==",
      "dev": true,
      "license": "Apache-2.0",
      "engines": {
        "node": "^20.19.0 || ^22.13.0 || >=24"
      }
    },
    "node_modules/@eslint/plugin-kit": {
      "version": "0.7.2",
      "resolved": "https://registry.npmjs.org/@eslint/plugin-kit/-/plugin-kit-0.7.2.tgz",
      "integrity": "sha512-+CNAzxglkrpNf/kKywqQfk74QjtceuOE7Qm+AF8miRvPF/wmmK5+OJOgVh3AVTT3RP2mH3+FO
k0A==",
      "dev": true,
      "license": "Apache-2.0",
      "dependencies": {
        "@eslint/core": "^1.2.1",
        "levn": "^0.4.1"
      },
      "engines": {
        "node": "^20.19.0 || ^22.13.0 || >=24"
      }
    },
    "node_modules/@humanfs/core": {
      "version": "0.19.2",
      "resolved": "https://registry.npmjs.org/@humanfs/core/-/core-0.19.2.tgz",
      "integrity": "sha512-UhXNm+CFMWcbChXywFwkmhqjs3PRCmcSa/hfBgLIb7oQ5HNb1wS0icWsGtSAUNgefHeI+eBrA
dvA==",
      "dev": true,
      "license": "Apache-2.0",
      "dependencies": {
        "@humanfs/types": "^0.15.0"
      },
      "engines": {
        "node": ">=18.18.0"
      }
    },
    "node_modules/@humanfs/node": {
      "version": "0.16.8",
      "resolved": "https://registry.npmjs.org/@humanfs/node/-/node-0.16.8.tgz",
      "integrity": "sha512-gE1eQNZ3R++kTzFUpdGlpmy8kDZD/MLyHqDwqjkVQI0JMdI1D51sy1H958PNXYkM2rAac7e5/
3BQ==",
      "dev": true,
      "license": "Apache-2.0",
      "dependencies": {
        "@humanfs/core": "^0.19.2",
```

```
        "@humanfs/types": "^0.15.0",
        "@humanwhocodes/retry": "^0.4.0"
      },
      "engines": {
        "node": ">=18.18.0"
      }
    },
    "node_modules/@humanfs/types": {
      "version": "0.15.0",
      "resolved": "https://registry.npmjs.org/@humanfs/types/-/types-0.15.0.tgz",
      "integrity": "sha512-ZZ1w0aoQkwuUuC7Yf+7sdeaNfqQiiLcSRbfI08oAxqLtpXQr9AIVX7Ay7HLDuiLYAaFPu8oBY
J3Q==",
      "dev": true,
      "license": "Apache-2.0",
      "engines": {
        "node": ">=18.18.0"
      }
    },
    "node_modules/@humanwhocodes/module-importer": {
      "version": "1.0.1",
      "resolved": "https://registry.npmjs.org/@humanwhocodes/module-importer/-/module-importer-1.0.1
      "integrity": "sha512-bxveV4V8v5Yb4ncFTT3rPSgZBOpCkjfK0y4oVVVJwIuDVBRMDXrPyXRL988i5ap9m9bnyEEjw
LfA==",
      "dev": true,
      "license": "Apache-2.0",
      "engines": {
        "node": ">=12.22"
      },
      "funding": {
        "type": "github",
        "url": "https://github.com/sponsors/nzakas"
      }
    },
    "node_modules/@humanwhocodes/retry": {
      "version": "0.4.3",
      "resolved": "https://registry.npmjs.org/@humanwhocodes/retry/-/retry-0.4.3.tgz",
      "integrity": "sha512-bV0Tgo9K4hfPCek+aMAn81RppFKv2ySDQeMoSZuvTASywNTnVJCArCZE2FWqpvIatKu7VMRLw
yhQ==",
      "dev": true,
      "license": "Apache-2.0",
      "engines": {
        "node": ">=18.18"
      },
      "funding": {
        "type": "github",
        "url": "https://github.com/sponsors/nzakas"
      }
    },
    "node_modules/@jridgewell/gen-mapping": {
      "version": "0.3.13",
      "resolved": "https://registry.npmjs.org/@jridgewell/gen-mapping/-/gen-mapping-0.3.13.tgz",
      "integrity": "sha512-2kkt/7niJ6MgEPxF0bYdQ6etZaA+fQvDcLKckhy1yIQOzaoKjBBjSj63/aLVjYE3qhRt5dvM+
BbA==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@jridgewell/sourcemap-codec": "^1.5.0",
        "@jridgewell/trace-mapping": "^0.3.24"
      }
    },
    "node_modules/@jridgewell/remapping": {
      "version": "2.3.5",
      "resolved": "https://registry.npmjs.org/@jridgewell/remapping/-/remapping-2.3.5.tgz",
      "integrity": "sha512-LI9u/+laYG4Ds1TDKSJW2YPrIlcVYOwi2fUC6xB43lueCjgxV4lffOCZCtYFiH6TNOX+tQKXx
HEQ==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
```

```
        "@jridgewell/gen-mapping": "^0.3.5",
        "@jridgewell/trace-mapping": "^0.3.24"
      }
    },
    "node_modules/@jridgewell/resolve-uri": {
      "version": "3.1.2",
      "resolved": "https://registry.npmjs.org/@jridgewell/resolve-uri/-/resolve-uri-3.1.2.tgz",
      "integrity": "sha512-bRISgCIjP20/tbWSPWMEi54QVPRZExkuD9lJL+UIxUKtwVJA8wW1Trb1jMs1RFXo1CBTNZ/5h
pKw==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=6.0.0"
      }
    },
    "node_modules/@jridgewell/sourcemap-codec": {
      "version": "1.5.5",
      "resolved": "https://registry.npmjs.org/@jridgewell/sourcemap-codec/-/sourcemap-codec-1.5.5.tg
      "integrity": "sha512-cYQ9310grqxueWbl+WuIUIaiUaDcj7WOq5fVhEljNVgRfOUhY9fy2zTvfoqWsnebh8Sl70VSc
0Og==",
      "dev": true,
      "license": "MIT"
    },
    "node_modules/@jridgewell/trace-mapping": {
      "version": "0.3.31",
      "resolved": "https://registry.npmjs.org/@jridgewell/trace-mapping/-/trace-mapping-0.3.31.tgz",
      "integrity": "sha512-zzNR+SdQSDJzc8joaeP8QQoCQr8NuYx2dIIytl1QeBEZHJ9uW6hebsrYgbz8hJwUQao3TWCMt
LAw==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@jridgewell/resolve-uri": "^3.1.0",
        "@jridgewell/sourcemap-codec": "^1.4.14"
      }
    },
    "node_modules/@napi-rs/wasm-runtime": {
      "version": "1.1.5",
      "resolved": "https://registry.npmjs.org/@napi-rs/wasm-runtime/-/wasm-runt
```

frontend/public/favicon.svg

```
<svg xmlns="http://www.w3.org/2000/svg" width="48" height="46" fill="none" viewBox="0 0 48 46"><path
3bff" d="M25.946 44.938c-.664.845-2.021.375-2.021-.698V33.937a2.26 2.26 0 0 0-2.262-2.262H10.287c-.9
1.04-.92-1.788l7.48-10.471c1.07-1.497 0-3.578-1.842-3.578H1.237c-.92 0-1.456-1.04-.92-1.788L10.013.4
97.556-.474.92-.474h28.894c.92 0 1.456 1.04.92 1.788l-7.48 10.471c-1.07 1.498 0 3.579 1.842 3.579h11
0 1.473 1.088.89 1.83L25.947 44.94z" style="fill:#863bff;fill:color(display-p3 .5252 .23 1);fill-opa
mask id="a" width="48" height="46" x="0" y="0" maskUnits="userSpaceOnUse" style="mask-type:alpha"><p
#000" d="M25.842 44.938c-.664.844-2.021.375-2.021-.698V33.937a2.26 2.26 0 0 0-2.262-2.262H10.183c-.9
1.04-.92-1.788l7.48-10.471c1.07-1.498 0-3.579-1.842-3.579H1.133c-.92 0-1.456-1.04-.92-1.787L9.91.473
.556-.474.92-.474h28.894c.92 0 1.456 1.04.92 1.788l-7.48 10.471c-1.07 1.498 0 3.578 1.842 3.578h11.3
1.473 1.088.89 1.832L25.843 44.94z" style="fill:#000;fill-opacity:1"/></mask><g mask="url(#a)"><g fi
#b)"><ellipse cx="5.508" cy="14.704" fill="#ede6ff" rx="5.508" ry="14.704" style="fill:#ede6ff;fill:
lay-p3 .9275 .9033 1);fill-opacity:1" transform="matrix(.00324 1 1 -.00324 -4.47 31.516)"/></g><g fi
#c)"><ellipse cx="10.399" cy="29.851" fill="#ede6ff" rx="10.399" ry="29.851" style="fill:#ede6ff;fil
splay-p3 .9275 .9033 1);fill-opacity:1" transform="matrix(.00324 1 1 -.00324 -39.328 7.883)"/></g><g
rl(#d)"><ellipse cx="5.508" cy="30.487" fill="#7e14ff" rx="5.508" ry="30.487" style="fill:#7e14ff;fi
isplay-p3 .4922 .0767 1);fill-opacity:1" transform="rotate(89.814 -25.913 -14.639)scale(1 -1)"/></g>
"url(#e)"><ellipse cx="5.508" cy="30.599" fill="#7e14ff" rx="5.508" ry="30.599" style="fill:#7e14ff;
(display-p3 .4922 .0767 1);fill-opacity:1" transform="rotate(89.814 -32.644 -3.334)scale(1 -1)"/></g
="url(#f)"><ellipse cx="5.508" cy="30.599" fill="#7e14ff" rx="5.508" ry="30.599" style="fill:#7e14ff
r(display-p3 .4922 .0767 1);fill-opacity:1" transform="matrix(.00324 1 1 -.00324 -34.34 30.47)"/></g
="url(#g)"><ellipse cx="14.072" cy="22.078" fill="#ede6ff" rx="14.072" ry="22.078" style="fill:#ede6
lor(display-p3 .9275 .9033 1);fill-opacity:1" transform="rotate(93.35 24.506 48.493)scale(-1 1)"/></
r="url(#h)"><ellipse cx="3.47" cy="21.501" fill="#7e14ff" rx="3.47" ry="21.501" style="fill:#7e14ff;
(display-p3 .4922 .0767 1);fill-opacity:1" transform="rotate(89.009 28.708 47.59)scale(-1 1)"/></g><
url(#i)"><ellipse cx="3.47" cy="21.501" fill="#7e14ff" rx="3.47" ry="21.501" style="fill:#7e14ff;fil
```

splay-p3 .4922 .0767 1);fill-opacity:1" transform="rotate(89.009 28.708 47.59)scale(-1 1)"/></g><g f
(#j)"><ellipse cx=".387" cy="8.972" fill="#7e14ff" rx="4.407" ry="29.108" style="fill:#7e14ff;fill:c
ay-p3 .4922 .0767 1);fill-opacity:1" transform="rotate(39.51 .387 8.972)"/></g><g filter="url(#k)"><
="47.523" cy="-6.092" fill="#7e14ff" rx="4.407" ry="29.108" style="fill:#7e14ff;fill:color(display-p
767 1);fill-opacity:1" transform="rotate(37.892 47.523 -6.092)"/></g><g filter="url(#l)"><ellipse cx
cy="6.333" fill="#47bfff" rx="5.971" ry="9.665" style="fill:#47bfff;fill:color(display-p3 .2799 .748
pacity:1" transform="rotate(37.892 41.412 6.333)"/></g><g filter="url(#m)"><ellipse cx="-1.879" cy="
ll="#7e14ff" rx="4.407" ry="29.108" style="fill:#7e14ff;fill:color(display-p3 .4922 .0767 1);fill-op
ransform="rotate(37.892 -1.88 38.332)"/></g><g filter="url(#n)"><ellipse cx="-1.879" cy="38.332" fil
" rx="4.407" ry="29.108" style="fill:#7e14ff;fill:color(display-p3 .4922 .0767 1);fill-opacity:1" tr
otate(37.892 -1.88 38.332)"/></g><g filter="url(#o)"><ellipse cx="35.651" cy="29.907" fill="#7e14ff"
" ry="29.108" style="fill:#7e14ff;fill:color(display-p3 .4922 .0767 1);fill-opacity:1" transform="ro
2 35.651 29.907)"/></g><g filter="url(#p)"><ellipse cx="38.418" cy="32.4" fill="#47bfff" rx="5.971"
" style="fill:#47bfff;fill:color(display-p3 .2799 .748 1);fill-opacity:1" transform="rotate(37.892 3
)"/></g></g><defs><filter id="b" width="60.045" height="41.654" x="-19.77" y="16.149" color-interpol
ers="sRGB" filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feB
ourceGraphic" in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlu
58" stdDeviation="7.659"/></filter><filter id="c" width="90.34" height="51.437" x="-54.613" y="-7.53
nterpolation-filters="sRGB" filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="Backgrou
"/><feBlend in="SourceGraphic" in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effe
oundBlur_2002_17158" stdDeviation="7.659"/></filter><filter id="d" width="79.355" height="29.4" x="-
2.03" color-interpolation-filters="sRGB" filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" res
roundImageFix"/><feBlend in="SourceGraphic" in2="BackgroundImageFix" result="shape"/><feGaussianBlur
ffect1_foregroundBlur_2002_17158" stdDeviation="4.596"/></filter><filter id="e" width="79.579" heigh
="-45.045" y="20.029" color-interpolation-filters="sRGB" filterUnits="userSpaceOnUse"><feFlood flood
0" result="BackgroundImageFix"/><feBlend in="SourceGraphic" in2="BackgroundImageFix" result="shape"/
anBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/></filter><filter id="f" widt
 height="29.4" x="-43.513" y="21.178" color-interpolation-filters="sRGB" filterUnits="userSpaceOnUse
 flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic" in2="BackgroundImageFix"
hape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/></filter><fi
" width="74.749" height="58.852" x="15.756" y="-17.901" color-interpolation-filters="sRGB" filterUni
aceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic" in2="B
mageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="7.
lter><filter id="h" width="61.377" height="25.362" x="23.548" y="2.284" color-interpolation-filters=
terUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="Sourc
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158"
on="4.596"/></filter><filter id="i" width="61.377" height="25.362" x="23.548" y="2.284" color-interp
lters="sRGB" filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><f
"SourceGraphic" in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundB
7158" stdDeviation="4.596"/></filter><filter id="j" width="56.045" height="63.649" x="-27.636" y="-2
or-interpolation-filters="sRGB" filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="Back
eFix"/><feBlend in="SourceGraphic" in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="
regroundBlur_2002_17158" stdDeviation="4.596"/></filter><filter id="k" width="54.814" height="64.646
6" y="-38.415" color-interpolation-filters="sRGB" filterUnits="userSpaceOnUse"><feFlood flood-opacit
lt="BackgroundImageFix"/><feBlend in="SourceGraphic" in2="BackgroundImageFix" result="shape"/><feGau
result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/></filter><filter id="l" width="33.5
="35.313" x="24.641" y="-11.323" color-interpolation-filters="sRGB" filterUnits="userSpaceOnUse"><fe
d-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic" in2="BackgroundImageFix" resu
/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/></filter><filter
th="54.814" height="64.646" x="-29.286" y="6.009" color-interpolation-filters="sRGB" filterUnits="us
se"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic" in2="Backgro
x" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/>
filter id="n" width="54.814" height="64.646" x="-29.286" y="6.009" color-interpolation-filters="sRGB
its="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGrap
BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDe
.596"/></filter><filter id="o" width="54.814" height="64.646" x="8.244" y="-2.416" color-interpolati
="sRGB" filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlen
ceGraphic" in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2
 stdDeviation="4.596"/></filter><filter id="p" width="39.409" height="43.623" x="18.713" y="10.588"
rpolation-filters="sRGB" filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundI
<feBlend in="SourceGraphic" in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1
dBlur_2002_17158" stdDeviation="4.596"/></filter></defs></svg>
```

### frontend/public/icons.svg

```
<svg xmlns="http://www.w3.org/2000/svg">
```
```

```
  <symbol id="bluesky-icon" viewBox="0 0 16 17">
    <g clip-path="url(#bluesky-clip)"><path fill="#08060d" d="M7.75 7.735c-.693-1.348-2.58-3.86-4.33
68-1.187-2.32-.981-2.74-.79C.188 2.065.1 2.812.1 3.251s.241 3.602.398 4.13c.52 1.744 2.367 2.333 4.0
495.37-4.71 1.278-1.805 4.512 3.196 3.309 4.38-.71 4.987-2.746.608 2.036 1.307 5.91 4.93 2.746 2.72-
4.143-1.747-4.512 1.702.189 3.55-.4 4.07-2.145.156-.528.397-3.691.397-4.13s-.088-1.186-.575-1.406c-.
6-.395-2.741.79-1.755 1.24-3.64 3.752-4.334 5.099"/></g>
    <defs><clipPath id="bluesky-clip"><path fill="#fff" d="M.1.85h15.3v15.3H.1z"/></clipPath></defs>
  </symbol>
  <symbol id="discord-icon" viewBox="0 0 20 19">
    <path fill="#08060d" d="M16.224 3.768a14.5 14.5 0 0 0-3.67-1.153c-.158.286-.343.67-.47.976a13.5
-4.067 0c-.128-.306-.317-.69-.476-.976A14.4 14.4 0 0 0 3.868 3.77C1.546 7.28.916 10.703 1.231 14.077
 0 0 0 4.5 2.306q.545-.748.965-1.587a9.5 9.5 0 0 1-1.518-.74q.191-.14.372-.293c2.927 1.369 6.107 1.3
q.183.152.372.294-.723.437-1.52.74.418.838.963 1.588a14.6 14.6 0 0 0 4.504-2.308c.37-3.911-.63-7.302
309m-9.13 8.234c-.878 0-1.599-.82-1.599-1.82 0-.998.705-1.82 1.6-1.82.894 0 1.614.82 1.599 1.82.001
2-1.6 1.82m5.91 0c-.878 0-1.599-.82-1.599-1.82 0-.998.705-1.82 1.6-1.82.893 0 1.614.82 1.599 1.82 0
2-1.6 1.82"/>
  </symbol>
  <symbol id="documentation-icon" viewBox="0 0 21 20">
    <path fill="none" stroke="#aa3bff" stroke-linecap="round" stroke-linejoin="round" stroke-width="
15.5 13.333 1.533 1.322c.645.555.967.833.967 1.178s-.322.623-.967 1.179L15.5 18.333m-3.333-5-1.534 1
.555-.966.833-.966 1.178s.322.623.966 1.179l1.534 1.321"/>
    <path fill="none" stroke="#aa3bff" stroke-linecap="round" stroke-linejoin="round" stroke-width="
17.167 10.836v-4.32c0-1.41 0-2.117-.224-2.68-.359-.906-1.118-1.621-2.08-1.96-.599-.21-1.349-.21-2.84
3 0-3.935 0-4.983.369-1.684.591-3.013 1.842-3.641 3.428C3 6.449 3 7.684 3 10.154v2.122c0 2.558 0 3.8
26q.306.383.713.671c.76.536 1.79.64 3.581.66"/>
    <path fill="none" stroke="#aa3bff" stroke-linecap="round" stroke-linejoin="round" stroke-width="
3 10a2.78 2.78 0 0 1 2.778-2.778c.555 0 1.209.097 1.748-.047.48-.129.854-.503.982-.982.145-.54.048-1
.749a2.78 2.78 0 0 1 2.777-2.777"/>
  </symbol>
  <symbol id="github-icon" viewBox="0 0 19 19">
    <path fill="#08060d" fill-rule="evenodd" d="M9.356 1.85C5.05 1.85 1.57 5.356 1.57 9.694a7.84 7.8
324 7.44c.387.079.528-.168.528-.376 0-.182-.013-.805-.013-1.454-2.165.467-2.616-.935-2.616-.935-.349
1.143-.864-1.143-.71-.48.051-.48.051-.48.787.051 1.2.805 1.2.805.695 1.194 1.817.857 2.268.649.064-.
7.49-1.052-1.728-.182-3.545-.857-3.545-3.87 0-.857.31-1.558.8-2.104-.078-.195-.349-1 .077-2.078 0 0
2.14.805a7.5 7.5 0 0 1 1.946-.26c.657 0 1.328.092 1.946.26 1.483-1.013 2.14-.805 2.14-.805.426 1.078
.078 2.078.502.546.799 1.247.799 2.104 0 3.013-1.818 3.675-3.558 3.87.284.247.528.714.528 1.454 0 1.
.896-.012 2.156 0 .208.142.455.528.377a7.84 7.84 0 0 0 5.324-7.441c.013-4.338-3.48-7.844-7.773-7.844
e="evenodd"/>
  </symbol>
  <symbol id="social-icon" viewBox="0 0 20 20">
    <path fill="none" stroke="#aa3bff" stroke-linecap="round" stroke-linejoin="round" stroke-width="
12.5 6.667a4.167 4.167 0 1 0-8.334 0 4.167 4.167 0 0 0 8.334 0"/>
    <path fill="none" stroke="#aa3bff" stroke-linecap="round" stroke-linejoin="round" stroke-width="
2.5 16.667a5.833 5.833 0 0 1 8.75-5.053m3.837.474.513 1.035c.07.144.257.282.414.309l.93.155c.596.1.7
.965l-.723.73a.64.64 0 0 0-.152.531l.207.903c.164.715-.213.991-.84.618l-.872-.52a.63.63 0 0 0-.577 0
-.624.373-1.003.094-.84-.618l.207-.903a.64.64 0 0 0-.152-.532l-.723-.729c-.426-.43-.289-.864.306-.96
a.64.64 0 0 0 .412-.31l.513-1.034c.28-.562.735-.562 1.012 0"/>
  </symbol>
  <symbol id="x-icon" viewBox="0 0 19 19">
    <path fill="#08060d" fill-rule="evenodd" d="M1.893 1.98c.052.072 1.245 1.769 2.653 3.77l2.892 4.
61.333.48.333.486s-.068.089-.152.183l-.522.593-.765.867-3.597 4.087c-.375.426-.734.834-.798.905a1 1
.148c0 .01.236.017.664.017h.663l.729-.83c.4-.457.796-.906.879-.999a692 692 0 0 0 1.794-2.038c.034-.0
.594-.675l.551-.624.345-.392a7 7 0 0 1 .34-.374c.006 0 .93 1.306 2.052 2.903l2.084 2.965.045.063h2.2
2.273-.003 2.266-.021-.008-.02-1.098-1.572-3.894-5.547-2.013-2.862-2.28-3.246-2.273-3.266.008-.019.2
085-2.38l2-2.274 1.567-1.782c.022-.028-.016-.03-.65-.03h-.674l-.3.342a871 871 0 0 1-1.782 2.025c-.06
.458-.75.852a100 100 0 0 1-.803.91c-.148.172-.299.344-.99 1.127-.304.343-.32.358-.345.327-.015-.019-
-1.976-2.808L6.365 1.85H1.8zm1.782.91 8.078 11.294c.772 1.08 1.413 1.973 1.425 1.984.016.017.241.02
.03-.004-2.694-3.766L7.796 5.75 5.722 2.852l-1.039-.004-1.039-.004z" clip-rule="evenodd"/>
  </symbol>
</svg>

```

frontend/README.md

```
# Frontend - Wi-Fi Billing System
```

This frontend is a starter React + TypeScript + Vite application for the Wi-Fi Billing System projec

The current UI is minimal and designed as a scaffold for building the admin portal, tenant dashboard
ent flow.

Project Structure

- `src/main.tsx` — React entrypoint that mounts the `App` component.
- `src/App.tsx` — Root application component and starter UI.
- `src/App.css` — Base styling for the starter page.
- `src/index.css` — Global CSS styles.
- `src/assets/` — Static images used by the starter page.

Dependencies

- `react`
- `react-dom`
- `vite`
- `typescript`
- `@vitejs/plugin-react`

Local Development

Install dependencies and start the development server:

```bash
cd frontend
npm install
npm run dev
```

Open the URL shown in the terminal to preview the app.

Build

Build the production bundle with:

```bash
npm run build
```

Next Frontend Steps

This frontend currently contains a placeholder page. Recommended next steps:

1. Add API client utilities to call the backend `/api/v1` endpoints.
2. Build auth flow using `/api/v1/auth/login`.
3. Add routing for admin, tenant, and customer dashboards.
4. Integrate payment initiation using `/api/v1/mpesa/stk-push`.
5. Create forms for tenant/router/customer management.

Notes

- The backend API is located under `backend/app/`.
- The frontend is not yet connected to backend logic.
- The current project is intended as a bootstrap for the Wi-Fi Billing SaaS platform.

```

### frontend/src/AdminDashboard.tsx

```
import React, { useState, useEffect } from 'react';

// --- TypeScript Interfaces ---
interface DashboardStats {
  total_revenue_kes: number;
  total_sales: number;

```
  active_users: number;
}

interface Transaction {
  id: string;
  amount: number;
  status: 'COMPLETED' | 'PENDING' | 'FAILED';
  checkout_request_id: string;
}

export default function AdminDashboard() {
  // Removed `useNavigate` to prevent the "<Router> component" error when previewing in isolation

  // --- Tab State ---
  const [activeTab, setActiveTab] = useState<'dashboard' | 'routers' | 'settings'>('dashboard');

  // --- Data State ---
  const [stats, setStats] = useState<DashboardStats | null>(null);
  const [transactions, setTransactions] = useState<Transaction[]>([]);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

  // Hardcoded for testing.
  const TENANT_ID = '11111111-1111-1111-1111-111111111111';

  // --- Data Fetching ---
  useEffect(() => {
    const fetchAdminData = async () => {
      try {
        const [statsRes, txRes] = await Promise.all([
          fetch(`http://localhost:8000/api/v1/admin/dashboard/stats/${TENANT_ID}`),
          fetch(`http://localhost:8000/api/v1/admin/transactions/${TENANT_ID}`)
        ]);

        if (!statsRes.ok || !txRes.ok) throw new Error('Failed to fetch dashboard data');

        const statsData = await statsRes.json();
        const txData = await txRes.json();

        setStats(statsData);
        // Ensure transactions is always an array to prevent rendering errors
        setTransactions(Array.isArray(txData.transactions) ? txData.transactions : []);
      } catch (err) {
        setError('Could not connect to the admin API. Ensure FastAPI is running.');
      } finally {
        setLoading(false);
      }
    };

    fetchAdminData();
    const interval = setInterval(fetchAdminData, 30000);
    return () => clearInterval(interval);
  }, []);

  const handleLogout = () => {
    sessionStorage.removeItem('isAdmin');
    // Using vanilla window.location avoids the need for a React Router context in standalone mode
    window.location.href = '/admin/login';
  };

  if (loading) return <div style={styles.center}>Loading Command Center...</div>;
  if (error) return <div style={{...styles.center, color: 'red'}}>{error}</div>;

  return (
    <div style={styles.container}>
      {/* --- SIDEBAR NAVIGATION --- */}
      <div style={styles.sidebar}>
        <h2 style={styles.brand}>Wi-Fi Manager</h2>
```

```
  <ul style={styles.nav}>
    <li
      style={activeTab === 'dashboard' ? styles.navItemActive : styles.navItem}
      onClick={() => setActiveTab('dashboard')}
    >
      Dashboard
    </li>
    <li
      style={activeTab === 'routers' ? styles.navItemActive : styles.navItem}
      onClick={() => setActiveTab('routers')}
    >
      Routers
    </li>
    <li
      style={activeTab === 'settings' ? styles.navItemActive : styles.navItem}
      onClick={() => setActiveTab('settings')}
    >
      Settings
    </li>
  </ul>
  <div style={{ marginTop: 'auto' }}>
    <button onClick={handleLogout} style={styles.logoutBtn}>Log Out</button>
  </div>
</div>

{/* --- MAIN CONTENT AREA --- */}
<div style={styles.main}>
  <div style={styles.header}>
    <h1 style={styles.title}>
      {activeTab === 'dashboard' && 'Overview'}
      {activeTab === 'routers' && 'Router Management'}
      {activeTab === 'settings' && 'System Settings'}
    </h1>
    {activeTab === 'dashboard' && (
      <button style={styles.refreshBtn} onClick={() => window.location.reload()}>
        ■ Refresh
      </button>
    )}
  </div>

  {/*  VIEW: DASHBOARD --- */}
  {activeTab === 'dashboard' && (
    <>
      <div style={styles.statsGrid}>
        <div style={styles.card}>
          <div style={styles.cardTitle}>Total Revenue</div>
          <div style={styles.cardValue}>KES {stats?.total_revenue_kes?.toLocaleString() || 0}<
        </div>
        <div style={styles.card}>
          <div style={styles.cardTitle}>Successful Sales</div>
          <div style={styles.cardValue}>{stats?.total_sales || 0}</div>
        </div>
        <div style={styles.card}>
          <div style={styles.cardTitle}>Active Connections</div>
          <div style={styles.cardValue}>{stats?.active_users || 0}</div>
        </div>
      </div>

      <div style={{...styles.card, marginTop: '24px'}}>
        <h2 style={{ fontSize: '18px', margin: '0 0 16px 0' }}>Recent Transactions</h2>
        <div style={styles.tableWrapper}>
          <table style={styles.table}>
            <thead>
              <tr>
                <th style={styles.th}>Checkout ID</th>
                <th style={styles.th}>Amount</th>
                <th style={styles.th}>Status</th>
              </tr>
```

```
                  </thead>
                  <tbody>
                    {transactions.map((tx) => (
                      <tr key={tx.id} style={styles.tr}>
                        <td style={styles.td}>
                          <span style={styles.code}>{tx.checkout_request_id.replace('ws_CO_', '...')
                        </td>
                        <td style={{...styles.td, fontWeight: '600'}}>KES {tx.amount}</td>
                        <td style={styles.td}>
                          <span style={{
                            ...styles.badge,
                            backgroundColor: tx.status === 'COMPLETED' ? '#def7ec' : tx.status === '
'#fde8e8' : '#fef3c7',
                            color: tx.status === 'COMPLETED' ? '#03543f' : tx.status === 'FAILED' ?
: '#92400e'
                          }}>
                            {tx.status}
                          </span>
                        </td>
                      </tr>
                    ))}
                    {transactions.length === 0 && (
                      <tr><td colSpan={3} style={{textAlign: 'center', padding: '20px', color: '#888
nsactions yet.</td></tr>
                    )}
                  </tbody>
                </table>
              </div>
            </div>
          </>
        )}

        {/* --- VIEW: ROUTERS --- */}
        {activeTab === 'routers' && (
          <div style={styles.card}>
            <h2 style={{ fontSize: '18px', margin: '0 0 16px 0' }}>MikroTik Integrations</h2>
            <p style={{ color: '#6b7280', fontSize: '14px', lineHeight: '1.5' }}>
              This panel will manage your active MikroTik routers. You will be able to see CPU usage
otspot users, and manually authorize or kick MAC addresses from here.
            </p>
            <button style={{...styles.refreshBtn, marginTop: '16px', backgroundColor: '#111827', col
}}>
              + Add New Router
            </button>
          </div>
        )}

        {/* --- VIEW: SETTINGS --- */}
        {activeTab === 'settings' && (
          <div style={styles.card}>
            <h2 style={{ fontSize: '18px', margin: '0 0 16px 0' }}>Tenant Configuration</h2>
            <p style={{ color: '#6b7280', fontSize: '14px', lineHeight: '1.5' }}>
              Configure your Daraja API keys, update internet package pricing, and manage your admin
ls here.
            </p>
          </div>
        )}
      </div>
    </div>
  );
}

// --- Styles ---
const styles: { [key: string]: React.CSSProperties } = {
  center: { display: 'flex', justifyContent: 'center', alignItems: 'center', height: '100vh', fontFa
s-serif' },
  container: { display: 'flex', minHeight: '100vh', backgroundColor: '#f3f4f6', fontFamily: '-apple-
inkMacSystemFont, "Segoe UI", Roboto, sans-serif' },
```

```
  sidebar: { width: '250px', backgroundColor: '#111827', color: '#fff', padding: '24px', display: 'f
Direction: 'column' },
  brand: { fontSize: '20px', fontWeight: '800', letterSpacing: '1px', marginBottom: '40px', color: '
  nav: { listStyle: 'none', padding: 0, margin: 0 },
  navItem: { padding: '12px 16px', borderRadius: '8px', marginBottom: '8px', cursor: 'pointer', colo
f', transition: 'all 0.2s' },
  navItemActive: { padding: '12px 16px', borderRadius: '8px', marginBottom: '8px', cursor: 'pointer'
ndColor: '#1f2937', color: '#fff', fontWeight: '600' },
  logoutBtn: { width: '100%', padding: '12px', backgroundColor: '#7f1d1d', color: '#fca5a5', border:
orderRadius: '8px', cursor: 'pointer', fontWeight: '600' },
  main: { flex: 1, padding: '40px', overflowY: 'auto' },
  header: { display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '3
  title: { fontSize: '28px', fontWeight: '700', color: '#111827', margin: 0 },
  refreshBtn: { padding: '8px 16px', backgroundColor: '#fff', border: '1px solid #d1d5db', borderRad
, cursor: 'pointer', fontWeight: '600', color: '#374151' },
  statsGrid: { display: 'grid', gridTemplateColumns: 'repeat(3, 1fr)', gap: '24px' },
  card: { backgroundColor: '#fff', borderRadius: '12px', padding: '24px', boxShadow: '0 1px 3px rgba
)' },
  cardTitle: { fontSize: '14px', color: '#6b7280', fontWeight: '600', marginBottom: '8px', textTrans
ercase', letterSpacing: '0.5px' },
  cardValue: { fontSize: '32px', fontWeight: '800', color: '#111827' },
  tableWrapper: { overflowX: 'auto' },
  table: { width: '100%', borderCollapse: 'collapse', textAlign: 'left' },
  th: { padding: '12px 16px', borderBottom: '1px solid #e5e7eb', color: '#6b7280', fontSize: '12px',
form: 'uppercase', fontWeight: '600' },
  tr: { borderBottom: '1px solid #f3f4f6' },
  td: { padding: '16px', fontSize: '14px', color: '#374151', verticalAlign: 'middle' },
  code: { fontFamily: 'monospace', backgroundColor: '#f3f4f6', padding: '4px 8px', borderRadius: '4p
ze: '13px' },
  badge: { padding: '4px 10px', borderRadius: '9999px', fontSize: '12px', fontWeight: '700', display
block' }
};
```

frontend/src/AdminLogin.tsx

```
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';

export default function AdminLogin() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const navigate = useNavigate();

  const handleLogin = (e: React.FormEvent) => {
    e.preventDefault();

    // Hardcoded credentials for testing (Change these later!)
    if (username === 'tesla' && password === 'admin123') {
      sessionStorage.setItem('isAdmin', 'true'); // Create the secure session
      navigate('/admin'); // Redirect to the dashboard
    } else {
      setError('Invalid username or password.');
    }
  };

  return (
    <div style={styles.container}>
      <div style={styles.card}>
        <h2 style={styles.title}>System Access</h2>
        <p style={styles.subtitle}>Enter your credentials to manage the network.</p>

        <form onSubmit={handleLogin} style={styles.form}>
          <div style={styles.inputGroup}>
            <label style={styles.label}>Username</label>
            <input
```

```
              type="text"
              value={username}
              onChange={(e) => setUsername(e.target.value)}
              style={styles.input}
              required
              autoComplete="off"
            />
          </div>

          <div style={styles.inputGroup}>
            <label style={styles.label}>Password</label>
            <input
              type="password"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              style={styles.input}
              required
            />
          </div>

          {error && <div style={styles.error}>{error}</div>}

          <button type="submit" style={styles.button}>Secure Login</button>
        </form>
      </div>
    </div>
  );
}

const styles: { [key: string]: React.CSSProperties } = {
  container: { display: 'flex', justifyContent: 'center', alignItems: 'center', minHeight: '100vh',
Color: '#111827', fontFamily: '-apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif' },
  card: { backgroundColor: '#1f2937', padding: '40px', borderRadius: '12px', width: '100%', maxWidth
 boxShadow: '0 10px 25px rgba(0,0,0,0.5)' },
  title: { color: '#fff', fontSize: '24px', margin: '0 0 8px 0', textAlign: 'center' },
  subtitle: { color: '#9ca3af', fontSize: '14px', margin: '0 0 24px 0', textAlign: 'center' },
  form: { display: 'flex', flexDirection: 'column', gap: '20px' },
  inputGroup: { display: 'flex', flexDirection: 'column', gap: '6px' },
  label: { color: '#e5e7eb', fontSize: '13px', fontWeight: '600' },
  input: { padding: '12px', borderRadius: '6px', border: '1px solid #374151', backgroundColor: '#374
r: '#fff', fontSize: '15px', outline: 'none' },
  button: { padding: '12px', borderRadius: '6px', border: 'none', backgroundColor: '#3b82f6', color:
ontSize: '15px', fontWeight: 'bold', cursor: 'pointer', marginTop: '10px' },
  error: { color: '#fca5a5', fontSize: '13px', textAlign: 'center', backgroundColor: '#7f1d1d', padd
, borderRadius: '6px' }
};
```

frontend/src/Admin.tsx

```
import React, { useEffect, useState } from 'react';

interface Transaction {
  id: string;
  phone_number: string;
  amount: number;
  status: string;
  created_at: string;
}

export default function Admin() {
  const [transactions, setTransactions] = useState<Transaction[]>([]);
  const [loading, setLoading] = useState<boolean>(true);

  useEffect(() => {
    // In production, this endpoint would require JWT authentication
    const fetchTransactions = async () => {
```

```
      try {
        const response = await fetch('http://localhost:8000/api/v1/transactions/');
        if (response.ok) {
          const data = await response.json();
          setTransactions(data);
        }
      } catch (error) {
        console.error("Failed to fetch transactions:", error);
      } finally {
        setLoading(false);
      }
    };

    fetchTransactions();
  }, []);

  return (
    <div style={{ padding: '40px', maxWidth: '1000px', margin: '0 auto' }}>
      <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBo
x' }}>
        <h1 style={{ margin: 0, color: '#111' }}>Dashboard Overview</h1>
        <div style={{ backgroundColor: '#e1effe', color: '#1e429f', padding: '8px 16px', borderRadiu
 fontWeight: 'bold' }}>
          System Status: ONLINE
        </div>
      </div>

      <div style={{ backgroundColor: '#fff', padding: '24px', borderRadius: '12px', boxShadow: '0 2p
a(0,0,0,0.05)' }}>
        <h2 style={{ fontSize: '18px', marginTop: 0, marginBottom: '20px', color: '#333' }}>Recent M
sactions</h2>

        {loading ? (
          <p>Loading data...</p>
        ) : (
          <table style={{ width: '100%', borderCollapse: 'collapse', textAlign: 'left' }}>
            <thead>
              <tr style={{ borderBottom: '2px solid #eee', color: '#666' }}>
                <th style={{ padding: '12px 8px' }}>Date</th>
                <th style={{ padding: '12px 8px' }}>Phone Number</th>
                <th style={{ padding: '12px 8px' }}>Amount (KES)</th>
                <th style={{ padding: '12px 8px' }}>Status</th>
              </tr>
            </thead>
            <tbody>
              {transactions.length === 0 ? (
                <tr>
                  <td colSpan={4} style={{ padding: '20px', textAlign: 'center', color: '#999' }}>No
ons found.</td>
                </tr>
              ) : (
                transactions.map((tx) => (
                  <tr key={tx.id} style={{ borderBottom: '1px solid #eee' }}>
                    <td style={{ padding: '12px 8px' }}>{new Date(tx.created_at).toLocaleString()}</
                    <td style={{ padding: '12px 8px' }}>{tx.phone_number}</td>
                    <td style={{ padding: '12px 8px', fontWeight: 'bold' }}>{tx.amount}</td>
                    <td style={{ padding: '12px 8px' }}>
                      <span style={{
                        backgroundColor: tx.status === 'COMPLETED' ? '#def7ec' : '#fde8e8',
                        color: tx.status === 'COMPLETED' ? '#03543f' : '#9b1c1c',
                        padding: '4px 8px',
                        borderRadius: '4px',
                        fontSize: '12px',
                        fontWeight: 'bold'
                      }}>
                        {tx.status}
                      </span>
                    </td>
```

```
                </tr>
              ))
            )}
          </tbody>
        </table>
      )}
    </div>
  </div>
  );
}
```

frontend/src/App.css

```
.counter {
  font-size: 16px;
  padding: 5px 10px;
  border-radius: 5px;
  color: var(--accent);
  background: var(--accent-bg);
  border: 2px solid transparent;
  transition: border-color 0.3s;
  margin-bottom: 24px;

  &:hover {
    border-color: var(--accent-border);
  }
  &:focus-visible {
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
}

.hero {
  position: relative;

  .base,
  .framework,
  .vite {
    inset-inline: 0;
    margin: 0 auto;
  }

  .base {
    width: 170px;
    position: relative;
    z-index: 0;
  }

  .framework,
  .vite {
    position: absolute;
  }

  .framework {
    z-index: 1;
    top: 34px;
    height: 28px;
    transform: perspective(2000px) rotateZ(300deg) rotateX(44deg) rotateY(39deg)
      scale(1.4);
  }

  .vite {
    z-index: 0;
    top: 107px;
    height: 26px;
    width: auto;
```

```
    transform: perspective(2000px) rotateZ(300deg) rotateX(40deg) rotateY(39deg)
      scale(0.8);
  }
}

#center {
  display: flex;
  flex-direction: column;
  gap: 25px;
  place-content: center;
  place-items: center;
  flex-grow: 1;

  @media (max-width: 1024px) {
    padding: 32px 20px 24px;
    gap: 18px;
  }
}

#next-steps {
  display: flex;
  border-top: 1px solid var(--border);
  text-align: left;

  & > div {
    flex: 1 1 0;
    padding: 32px;
    @media (max-width: 1024px) {
      padding: 24px 20px;
    }
  }

  .icon {
    margin-bottom: 16px;
    width: 22px;
    height: 22px;
  }

  @media (max-width: 1024px) {
    flex-direction: column;
    text-align: center;
  }
}

#docs {
  border-right: 1px solid var(--border);

  @media (max-width: 1024px) {
    border-right: none;
    border-bottom: 1px solid var(--border);
  }
}

#next-steps ul {
  list-style: none;
  padding: 0;
  display: flex;
  gap: 8px;
  margin: 32px 0 0;

  .logo {
    height: 18px;
  }

  a {
    color: var(--text-h);
    font-size: 16px;
    border-radius: 6px;
```

```
    background: var(--social-bg);
    display: flex;
    padding: 6px 12px;
    align-items: center;
    gap: 8px;
    text-decoration: none;
    transition: box-shadow 0.3s;

    &:hover {
      box-shadow: var(--shadow);
    }
    .button-icon {
      height: 18px;
      width: 18px;
    }
  }

  @media (max-width: 1024px) {
    margin-top: 20px;
    flex-wrap: wrap;
    justify-content: center;

    li {
      flex: 1 1 calc(50% - 8px);
    }

    a {
      width: 100%;
      justify-content: center;
      box-sizing: border-box;
    }
  }
}

#spacer {
  height: 88px;
  border-top: 1px solid var(--border);
  @media (max-width: 1024px) {
    height: 48px;
  }
}

.ticks {
  position: relative;
  width: 100%;

  &::before,
  &::after {
    content: '';
    position: absolute;
    top: -4.5px;
    border: 5px solid transparent;
  }

  &::before {
    left: 0;
    border-left-color: var(--border);
  }
  &::after {
    right: 0;
    border-right-color: var(--border);
  }
}

```

frontend/src/App.tsx

```
import React from 'react';
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import CaptivePortal from './CaptivePortal';
import AdminDashboard from './AdminDashboard';
import AdminLogin from './AdminLogin'; // Import the new login screen

//component checks the session before rendering the dashboard
const ProtectedRoute = ({ children }: { children: JSX.Element }) => {
  const isAuthenticated = sessionStorage.getItem('isAdmin') === 'true';

  if (!isAuthenticated) {
    // If they aren't logged in, kick them back to the login page
    return <Navigate to="/admin/login" replace />;
  }

  return children;
};

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        {/* PUBLIC ROUTE: The Captive Portal for customers */}
        <Route path="/" element={<CaptivePortal />} />

        {/* PUBLIC ROUTE: The Admin Login Screen */}
        <Route path="/admin/login" element={<AdminLogin />} />

        {/* SECURE ROUTE: The actual dashboard, wrapped in our protection logic */}
        <Route
          path="/admin"
          element={
            <ProtectedRoute>
              <AdminDashboard />
            </ProtectedRoute>
          }
        />

        {/* Fallback route */}
        <Route path="*" element={<Navigate to="/" replace />} />
      </Routes>
    </BrowserRouter>
  );
}
```

frontend/src/assets/hero.png

binary or skipped file: .png

frontend/src/assets/react.svg

```
<svg xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://www.w3.org/1999/xlink" aria-hidden="true
g" class="iconify iconify--logos" width="35.93" height="32" preserveAspectRatio="xMidYMid meet" view
56 228"><path fill="#00D8FF" d="M210.483 73.824a171.49 171.49 0 0 0-8.24-2.597c.465-1.9.893-3.777 1.
6.238-30.281 2.16-54.676-11.769-62.708c-13.355-7.7-35.196.329-57.254 19.526a171.23 171.23 0 0 0-6.37
5.866 155.866 0 0 0-4.241-3.917C100.759 3.829 77.587-4.822 63.673 3.233C50.33 10.957 46.379 33.89 51
8a170.974 170.974 0 0 0 1.892 8.48c-3.28.932-6.445 1.924-9.474 2.98C17.309 83.498 0 98.307 0 113.668
18.582 31.778 46.812 41.427a145.52 145.52 0 0 0 6.921 2.165a167.467 167.467 0 0 0-2.01 9.138c-5.354
 50.591 12.134 58.266c13.744 7.926 36.812-.22 59.273-19.855a145.567 145.567 0 0 0 5.342-4.923a168.06
0 0 0 6.92 6.314c21.758 18.722 43.246 26.282 56.54 18.586c13.731-7.949 18.194-32.003 12.4-61.268a145
16 0 0 0-1.535-6.842c1.62-.48 3.21-.974 4.76-1.488c29.348-9.723 48.443-25.443 48.443-41.52c0-15.417-
326-45.517-39.844Zm-6.365 70.984c-1.4.463-2.836.91-4.3 1.345c-3.24-10.257-7.612-21.163-12.963-32.432
9.31-21.767 12.459-31.957c2.619.758 5.16 1.557 7.61 2.4c23.69 8.156 38.14 20.213 38.14 29.504c0 9.89
2.743-40.946 31.14Zm-10.514 20.834c2.562 12.94 2.927 24.64 1.23 33.787c-1.524 8.219-4.59 13.698-8.38
8.067 4.67-25.32-1.4-43.927-17.412a156.726 156.726 0 0 1-6.437-5.87c7.214-7.889 14.423-17.06 21.459-
```

376-1.098 24.068-2.894 34.671-5.345a134.17 134.17 0 0 1 1.386 6.193ZM87.276 214.515c-7.882 2.783-14.
7.955.675c-8.075-4.657-11.432-22.636-6.853-46.752a156.923 156.923 0 0 1 1.869-8.499c10.486 2.32 22.0
4.498 4.994c7.084 9.967 14.501 19.128 21.976 27.15a134.668 134.668 0 0 1-4.877 4.492c-9.933 8.682-19
2-28.658 17.94ZM50.35 144.747c-12.483-4.267-22.792-9.812-29.858-15.863c-6.35-5.437-9.555-10.836-9.55
-9.322 13.897-21.212 37.076-29.293c2.813-.98 5.757-1.905 8.812-2.773c3.204 10.42 7.406 21.315 12.477
.137 11.18-9.399 22.249-12.634 32.792a134.718 134.718 0 0 1-6.318-1.979Zm12.378-84.26c-4.811-24.587-
34 6.425-47.789c8.564-4.958 27.502 2.111 47.463 19.835a144.318 144.318 0 0 1 3.841 3.545c-7.438 7.98
7.08-21.808 26.988c-12.04 1.116-23.565 2.908-34.161 5.309a160.342 160.342 0 0 1-1.76-7.887Zm110.427
.8 347.8 0 0 0-7.785-12.803c8.168 1.033 15.994 2.404 23.343 4.08c-2.206 7.072-4.956 14.465-8.193 22.
1 381.151 0 0 0-7.365-13.322Zm-45.032-43.861c5.044 5.465 10.096 11.566 15.065 18.186a322.04 322.04 0
7-.006c4.974-6.559 10.069-12.652 15.192-18.18ZM82.802 87.83a323.167 323.167 0 0 0-7.227 13.238c-3.18
909-14.98-8.134-22.152c7.304-1.634 15.093-2.97 23.209-3.984a321.524 321.524 0 0 0-7.848 12.897Zm8.08
8.385-.936-16.291-2.203-23.593-3.793c2.26-7.3 5.045-14.885 8.298-22.6a321.187 321.187 0 0 0 7.257 13
 4.48 5.28 8.868 8.038 13.147Zm37.542 31.03c-5.184-5.592-10.354-11.779-15.403-18.433c4.902.192 9.899
.29c5.218 0 10.376-.117 15.453-.343c-4.985 6.774-10.018 12.97-15.028 18.486Zm52.198-57.817c3.422 7.8
345 8.596 22.52c-7.422 1.694-15.436 3.058-23.88 4.071a382.417 382.417 0 0 0 7.859-13.026a347.403 347
 7.425-13.565Zm-16.898 8.101a358.557 358.557 0 0 1-12.281 19.815a329.4 329.4 0 0 1-23.444.823c-7.967
.248-23.178-.732a310.202 310.202 0 0 1-12.513-19.846h.001a307.41 307.41 0 0 1-10.923-20.627a310.278
0 1 10.89-20.637l-.001.001a307.318 307.318 0 0 1 12.413-19.761c7.613-.576 15.42-.876 23.31-.876H128c
.743.303 23.354.883a329.357 329.357 0 0 1 12.335 19.695a358.489 358.489 0 0 1 11.036 20.54a329.472 3
 1-11 20.722Zm22.56-122.124c8.572 4.944 11.906 24.881 6.52 51.026c-.344 1.668-.73 3.367-1.15 5.09c-1
2-22.155-4.275-34.23-5.408c-7.034-10.017-14.323-19.124-21.64-27.008a160.789 160.789 0 0 1 5.888-5.4c
7 36.564-22.941 44.612-18.3ZM128 90.808c12.625 0 22.86 10.235 22.86 22.86s-10.235 22.86-22.86 22.86s
235-22.86-22.86s10.235-22.86 22.86-22.86Z"></path></svg>
```

### frontend/src/assets/vite.svg

```
<svg xmlns="http://www.w3.org/2000/svg" width="77" height="47" fill="none" aria-labelledby="vite-log
iewBox="0 0 77 47"><title id="vite-logo-title">Vite</title><style>.parenthesis{fill:#000}@media (pre
-scheme:dark){.parenthesis{fill:#fff}}</style><path fill="#9135ff" d="M40.151 45.71c-.663.844-2.02.3
99V34.708a2.26 2.26 0 0 0-2.262-2.262H24.493c-.92 0-1.457-1.04-.92-1.788l7.479-10.471c1.07-1.498 0-3
-3.578H15.443c-.92 0-1.456-1.04-.92-1.788l9.696-13.576c.213-.297.556-.474.92-.474h28.894c.92 0 1.456
.788l-7.48 10.472c-1.07 1.497 0 3.578 1.842 3.578h11.376c.944 0 1.474 1.087.89 1.83L40.153 45.712z"/
"a" width="48" height="47" x="14" y="0" maskUnits="userSpaceOnUse" style="mask-type:alpha"><path fil
="M40.047 45.71c-.663.843-2.02.374-2.02-.699V34.708a2.26 2.26 0 0 0-2.262-2.262H24.389c-.92 0-1.457-
.788l7.479-10.472c1.07-1.497 0-3.578-1.842-3.578H15.34c-.92 0-1.456-1.04-.92-1.788l9.696-13.575c.213
.474.92-.474H53.93c.92 0 1.456 1.04.92 1.788L47.37 13.03c-1.07 1.498 0 3.578 1.842 3.578h11.376c.944
.088.89 1.831L40.049 45.712z"/></mask><g mask="url(#a)"><g filter="url(#b)"><ellipse cx="5.508" cy="
ll="#eee6ff" rx="5.508" ry="14.704" transform="rotate(269.814 20.96 11.29)scale(-1 1)"/></g><g filte
"><ellipse cx="10.399" cy="29.851" fill="#eee6ff" rx="10.399" ry="29.851" transform="rotate(89.814 -
275)scale(1 -1)"/></g><g filter="url(#d)"><ellipse cx="5.508" cy="30.487" fill="#8900ff" rx="5.508"
" transform="rotate(89.814 -19.197 -7.127)scale(1 -1)"/></g><g filter="url(#e)"><ellipse cx="5.508"
" fill="#8900ff" rx="5.508" ry="30.599" transform="rotate(89.814 -25.928 4.177)scale(1 -1)"/></g><g
l(#f)"><ellipse cx="5.508" cy="30.599" fill="#8900ff" rx="5.508" ry="30.599" transform="rotate(89.81
5.52)scale(1 -1)"/></g><g filter="url(#g)"><ellipse cx="14.072" cy="22.078" fill="#eee6ff" rx="14.07
078" transform="rotate(93.35 31.245 55.578)scale(-1 1)"/></g><g filter="url(#h)"><ellipse cx="3.47"
" fill="#8900ff" rx="3.47" ry="21.501" transform="rotate(89.009 35.419 55.202)scale(-1 1)"/></g><g f
(#i)"><ellipse cx="3.47" cy="21.501" fill="#8900ff" rx="3.47" ry="21.501" transform="rotate(89.009 3
02)scale(-1 1)"/></g><g filter="url(#j)"><ellipse cx="14.592" cy="9.743" fill="#8900ff" rx="4.407" r
 transform="rotate(39.51 14.592 9.743)"/></g><g filter="url(#k)"><ellipse cx="61.728" cy="-5.321" fi
f" rx="4.407" ry="29.108" transform="rotate(37.892 61.728 -5.32)"/></g><g filter="url(#l)"><ellipse
" cy="7.104" fill="#00c2ff" rx="5.971" ry="9.665" transform="rotate(37.892 55.618 7.104)"/></g><g fi
#m)"><ellipse cx="12.326" cy="39.103" fill="#8900ff" rx="4.407" ry="29.108" transform="rotate(37.892
.103)"/></g><g filter="url(#n)"><ellipse cx="12.326" cy="39.103" fill="#8900ff" rx="4.407" ry="29.10
rm="rotate(37.892 12.326 39.103)"/></g><g filter="url(#o)"><ellipse cx="49.857" cy="30.678" fill="#8
"4.407" ry="29.108" transform="rotate(37.892 49.857 30.678)"/></g><g filter="url(#p)"><ellipse cx="5
"33.171" fill="#00c2ff" rx="5.971" ry="15.297" transform="rotate(37.892 52.623 33.17)"/></g></g><pat
9 0c-9.198 13.166-9.252 33.575 0 46.789h6.215c-9.25-13.214-9.196-33.623 0-46.789zm62.424 0h-6.215c9.
 9.252 33.575 0 46.789h6.215c9.25-13.214 9.196-33.623 0-46.789" class="parenthesis"/><defs><filter i
h="60.045" height="41.654" x="-5.564" y="16.92" color-interpolation-filters="sRGB" filterUnits="user
"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic" in2="Backgroun
 result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17286" stdDeviation="7.659"/></
lter id="c" width="90.34" height="51.437" x="-40.407" y="-6.762" color-interpolation-filters="sRGB"
s="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphi
ckgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17286" stdDevi
59"/></filter><filter id="d" width="79.355" height="29.4" x="-35.435" y="2.801" color-interpolation-
```
```

RGB" filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend i
raphic" in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002
dDeviation="4.596"/></filter><filter id="e" width="79.579" height="29.4" x="-30.84" y="20.8" color-i
on-filters="sRGB" filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix
d in="SourceGraphic" in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregr
002_17286" stdDeviation="4.596"/></filter><filter id="f" width="79.579" height="29.4" x="-29.307" y=
olor-interpolation-filters="sRGB" filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="Ba
ageFix"/><feBlend in="SourceGraphic" in2="BackgroundImageFix" result="shape"/><feGaussianBlur result
foregroundBlur_2002_17286" stdDeviation="4.596"/></filter><filter id="g" width="74.749" height="58.8
961" y="-17.13" color-interpolation-filters="sRGB" filterUnits="userSpaceOnUse"><feFlood flood-opaci
ult="BackgroundImageFix"/><feBlend in="SourceGraphic" in2="BackgroundImageFix" result="shape"/><feGa
result="effect1_foregroundBlur_2002_17286" stdDeviation="7.659"/></filter><filter id="h" width="61.
t="25.362" x="37.754" y="3.055" color-interpolation-filters="sRGB" filterUnits="userSpaceOnUse"><feF
-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic" in2="BackgroundImageFix" resul
><feGaussianBlur result="effect1_foregroundBlur_2002_17286" stdDeviation="4.596"/></filter><filter i
h="61.377" height="25.362" x="37.754" y="3.055" color-interpolation-filters="sRGB" filterUnits="user
"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic" in2="Backgroun
result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17286" stdDeviation="4.596"/></
lter id="j" width="56.045" height="63.649" x="-13.43" y="-22.082" color-interpolation-filters="sRGB"
ts="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraph
ackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17286" stdDev
596"/></filter><filter id="k" width="54.814" height="64.646" x="34.321" y="-37.644" color-interpolat
s="sRGB" filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBle
rceGraphic" in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_
" stdDeviation="4.596"/></filter><filter id="l" width="33.541" height="35.313" x="38.847" y="-10.552
terpolation-filters="sRGB" filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="Backgroun
/><feBlend in="SourceGraphic" in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effec
undBlur_2002_17286" stdDeviation="4.596"/></filter><filter id="m" width="54.814" height="64.646" x="
="6.78" color-interpolation-filters="sRGB" filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" r
kgroundImageFix"/><feBlend in="SourceGraphic" in2="BackgroundImageFix" result="shape"/><feGaussianBl
"effect1_foregroundBlur_2002_17286" stdDeviation="4.596"/></filter><filter id="n" width="54.814" hei
6" x="-15.081" y="6.78" color-interpolation-filters="sRGB" filterUnits="userSpaceOnUse"><feFlood flo
="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic" in2="BackgroundImageFix" result="shape
sianBlur result="effect1_foregroundBlur_2002_17286" stdDeviation="4.596"/></filter><filter id="o" wi
4" height="64.646" x="22.45" y="-1.645" color-interpolation-filters="sRGB" filterUnits="userSpaceOnU
od flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic" in2="BackgroundImageFi
"shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17286" stdDeviation="4.596"/></filter><
"p" width="39.409" height="43.623" x="32.919" y="11.36" color-interpolation-filters="sRGB" filterUni
aceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic" in2="B
mageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17286" stdDeviation="4.
lter></defs></svg>

```

### frontend/src/CaptivePortal.tsx

```
import React, { useState, useEffect } from 'react';

interface InternetPlan {
  id: string;
  name: string;
  price: number;
  duration: string;
}

export default function CaptivePortal() {
  // Navigation State now includes 'success'
  const [view, setView] = useState<'plans' | 'payment' | 'success'>('plans');

  // Data State
  const [selectedPlan, setSelectedPlan] = useState<InternetPlan | null>(null);
  const [phoneNumber, setPhoneNumber] = useState<string>('');
  const [checkoutId, setCheckoutId] = useState<string | null>(null); // Tracks the active M-Pesa req

  // UI State
  const [loading, setLoading] = useState<boolean>(false);
  const [statusMessage, setStatusMessage] = useState<{ type: 'success' | 'error' | 'info'; text: str
```
```

```
ll>(null);

  // Exact UUIDs matching your PostgreSQL database
  const plans: InternetPlan[] = [
    { id: 'a1111111-1111-1111-1111-111111111111', name: '1 Hour Quick', price: 10, duration: '1 Hour
    { id: 'a2222222-2222-2222-2222-222222222222', name: '4 Hours Pass', price: 20, duration: '4 Hour
    { id: 'a3333333-3333-3333-3333-333333333333', name: '24 Hours Daily', price: 40, duration: '24 H
    { id: 'a4444444-4444-4444-4444-444444444444', name: 'Weekly Pass', price: 200, duration: '7 Days
    { id: 'a5555555-5555-5555-5555-555555555555', name: 'Bi-Weekly', price: 300, duration: '14 Days'
    { id: 'a6666666-6666-6666-6666-666666666666', name: 'Monthly Max', price: 650, duration: '30 Day
  ];

  // --- THE POLLING ENGINE ---
  useEffect(() => {
    let intervalId: NodeJS.Timeout;

    // Only run the poll if we have a checkoutId and are actively on the payment screen
    if (checkoutId && view === 'payment') {
      intervalId = setInterval(async () => {
        try {
          // Ask FastAPI for the exact status of this specific transaction
          const res = await fetch(`http://localhost:8000/api/v1/mpesa/status/${checkoutId}`);

          if (res.ok) {
            const data = await res.json();

            if (data.status === 'COMPLETED') {
              setCheckoutId(null); // Stop polling
              setLoading(false);
              setView('success');  // Transition to the connected screen
            } else if (data.status === 'FAILED' || data.status === 'CANCELLED') {
              setCheckoutId(null); // Stop polling
              setLoading(false);
              setStatusMessage({ type: 'error', text: 'Payment failed or was cancelled. Please try a
            }
            // If status is 'PENDING', the loop just quietly continues on the next tick
          }
        } catch (error) {
          console.error("Polling error: Lost connection to backend.", error);
        }
      }, 3000); // Ping the backend every 3 seconds
    }

    // Cleanup function: If the component unmounts or state changes, kill the timer immediately to p
ory leaks.
    return () => clearInterval(intervalId);
  }, [checkoutId, view]);

  const handlePlanClick = (plan: InternetPlan) => {
    setSelectedPlan(plan);
    setStatusMessage(null);
    setView('payment');
  };

  const handleBackToPlans = () => {
    setView('plans');
    setStatusMessage(null);
    setCheckoutId(null); // Abort polling if they back out
  };

  const handlePayment = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!selectedPlan) return;

    let formattedPhone = phoneNumber.replace('+', '').trim();
    if (formattedPhone.startsWith('0')) {
      formattedPhone = '254' + formattedPhone.slice(1);
    }
```

```
    if (!formattedPhone.match(/^254(7|1)\d{8}$/)) {
      setStatusMessage({ type: 'error', text: 'Enter a valid Safaricom number starting with 07 or 01
      return;
    }

    setLoading(true);
    setStatusMessage({ type: 'info', text: 'Sending M-Pesa STK Push... Check your phone.' });

    try {
      const response = await fetch('http://localhost:8000/api/v1/mpesa/stk-push', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          phone_number: formattedPhone,
          plan_id: selectedPlan.id,
          tenant_id: '11111111-1111-1111-1111-111111111111'
        }),
      });

      const data = await response.json();

      if (response.ok) {
        setStatusMessage({ type: 'success', text: 'STK Push sent! Please enter your M-Pesa PIN on yo
});
        // Start the polling engine by setting the ID returned from Safaricom
        setCheckoutId(data.checkout_id);
      } else {
        let errorMessage = 'Failed to initiate payment. Please try again.';
        if (data.detail) {
          if (typeof data.detail === 'string') {
            errorMessage = data.detail;
          } else if (Array.isArray(data.detail)) {
            errorMessage = `Validation Error: ${data.detail[0].loc.join(' -> ')} - ${data.detail[0].
          }
        }
        setStatusMessage({ type: 'error', text: errorMessage });
        setLoading(false);
      }
    } catch (error) {
      setStatusMessage({ type: 'error', text: 'Cannot connect to backend server. Ensure the backend
.' });
      setLoading(false);
    }
  };

  return (
    <div style={styles.container}>
      <div style={styles.card}>

        {/* VIEW 1: PLANS */}
        {view === 'plans' && (
          <div>
            <div style={styles.header}>
              <h1 style={styles.title}>HIGH-SPEED WI-FI</h1>
              <p style={styles.subtitle}>Select a plan to get instant internet access.</p>
            </div>
            <div style={styles.planGrid}>
              {plans.map((plan) => (
                <div
                  key={plan.id}
                  onClick={() => handlePlanClick(plan)}
                  style={styles.planCard}
                >
                  <div style={styles.planName}>{plan.name}</div>
                  <div style={styles.planDuration}>{plan.duration}</div>
                  <div style={styles.planPrice}>KES {plan.price}</div>
                </div>
```

```
            ))}
          </div>
        </div>
      )}

      {/* VIEW 2: PAYMENT */}
      {view === 'payment' && selectedPlan && (
        <div>
          <button onClick={handleBackToPlans} style={styles.backButton} disabled={loading}>
            ← Back to Packages
          </button>

          <div style={styles.header}>
            <h1 style={styles.title}>Payment Checkout</h1>
          </div>

          <div style={styles.summaryCard}>
            <span style={styles.summaryTitle}>{selectedPlan.name}</span>
            <span style={styles.summaryPrice}>KES {selectedPlan.price}</span>
          </div>

          <form onSubmit={handlePayment} style={styles.form}>
            <div style={styles.inputGroup}>
              <label style={styles.label}>M-Pesa Phone Number</label>
              <input
                type="tel"
                placeholder="e.g., 0712345678 or 0112345678"
                value={phoneNumber}
                onChange={(e) => setPhoneNumber(e.target.value)}
                disabled={loading}
                style={styles.input}
                required
              />
            </div>

            {statusMessage && (
              <div style={{
                ...styles.alert,
                backgroundColor: statusMessage.type === 'error' ? '#fde8e8' : statusMessage.type =
s' ? '#def7ec' : '#e1effe',
                color: statusMessage.type === 'error' ? '#9b1c1c' : statusMessage.type === 'succes
43f' : '#1e429f'
              }}>
                {statusMessage.text}
              </div>
            )}

            <button
              type="submit"
              disabled={loading}
              style={{
                ...styles.button,
                backgroundColor: loading ? '#a0a0a0' : '#000',
                cursor: loading ? 'not-allowed' : 'pointer'
              }}
            >
              {loading ? 'Waiting for Payment...' : 'Pay & Connect Now'}
            </button>
          </form>
        </div>
      )}

      {/* VIEW 3: SUCCESS (Auto-Connection) */}
      {view === 'success' && (
        <div style={{ textAlign: 'center', padding: '20px 0' }}>
          <div style={{ fontSize: '48px', marginBottom: '16px' }}>■</div>
          <h1 style={{ color: '#03543f', margin: '0 0 12px 0', fontSize: '24px', fontWeight: '800'
t Received!</h1>
```

```
            <p style={{ color: '#333', fontSize: '15px', lineHeight: '1.5', margin: '0 0 24px 0' }}>
              Your device has been authorized on the network. You are now securely connected to the
            </p>
            <button
              onClick={() => {
                setView('plans');
                setPhoneNumber('');
              }}
              style={{...styles.button, width: '100%'}}
            >
              Return Home
            </button>
          </div>
        )}

        <div style={styles.footer}>
          Powered by Standalone Billing Engine
        </div>
      </div>
    </div>
  );
}

const styles: { [key: string]: React.CSSProperties } = {
  container: { display: 'flex', justifyContent: 'center', alignItems: 'center', minHeight: '100vh',
Color: '#f5f5f7', fontFamily: '-apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif', p
0px' },
  card: { backgroundColor: '#fff', borderRadius: '16px', boxShadow: '0 4px 20px rgba(0, 0, 0, 0.05)'
100%', maxWidth: '440px', padding: '32px 24px', boxSizing: 'border-box' },
  header: { textAlign: 'center', marginBottom: '28px' },
  title: { fontSize: '24px', fontWeight: '800', letterSpacing: '1px', margin: '0 0 8px 0', color: '#
  subtitle: { fontSize: '14px', color: '#666', margin: 0, lineHeight: '1.4' },
  planGrid: { display: 'grid', gridTemplateColumns: 'repeat(2, 1fr)', gap: '12px' },
  planCard: { border: '2px solid #e0e0e0', borderRadius: '12px', padding: '16px 8px', textAlign: 'ce
sor: 'pointer', transition: 'all 0.2s ease', backgroundColor: '#fff' },
  planName: { fontSize: '13px', fontWeight: '600', color: '#333', marginBottom: '4px' },
  planDuration: { fontSize: '11px', color: '#888', marginBottom: '8px' },
  planPrice: { fontSize: '15px', fontWeight: '800', color: '#000' },
  backButton: { background: 'none', border: 'none', color: '#1e429f', fontSize: '14px', fontWeight:
sor: 'pointer', padding: 0, marginBottom: '20px' },
  summaryCard: { display: 'flex', justifyContent: 'space-between', alignItems: 'center', padding: '1
groundColor: '#f9fafb', borderRadius: '8px', border: '1px solid #e5e7eb', marginBottom: '24px' },
  summaryTitle: { fontSize: '15px', fontWeight: '600', color: '#111' },
  summaryPrice: { fontSize: '16px', fontWeight: '800', color: '#1e429f' },
  form: { display: 'flex', flexDirection: 'column', gap: '20px' },
  inputGroup: { display: 'flex', flexDirection: 'column', gap: '6px' },
  label: { fontSize: '13px', fontWeight: '600', color: '#333' },
  input: { padding: '12px 16px', borderRadius: '8px', border: '1px solid #ccc', fontSize: '16px', ou
ne', boxSizing: 'border-box', width: '100%' },
  button: { color: '#fff', border: 'none', borderRadius: '8px', padding: '14px', fontSize: '16px', f
 '600', transition: 'background-color 0.2s ease', marginTop: '8px', width: '100%' },
  alert: { padding: '12px', borderRadius: '8px', fontSize: '13px', lineHeight: '1.4', textAlign: 'ce
  footer: { textAlign: 'center', fontSize: '11px', color: '#aaa', marginTop: '32px' }
};
```

frontend/src/index.css

```
body, html {
  margin: 0;
  padding: 0;
  width: 100%;
  height: 100%;
  background-color: #f5f5f7;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
```

````
}

* {
  box-sizing: border-box;
}

#root {
  width: 100%;
  height: 100%;
}
```

### frontend/src/main.tsx

```
// src/main.tsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App' // This is our Traffic Controller
import './index.css'

ReactDOM.createRoot(document.getElementById('root') as HTMLElement).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

### frontend/tsconfig.app.json

```
{
  "compilerOptions": {
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.app.tsbuildinfo",
    "target": "es2023",
    "lib": ["ES2023", "DOM"],
    "module": "esnext",
    "types": ["vite/client"],
    "skipLibCheck": true,

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "moduleDetection": "force",
    "noEmit": true,
    "jsx": "react-jsx",

    /* Linting */
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src"]
}

```

### frontend/tsconfig.json

```
{
  "files": [],
  "references": [
    { "path": "./tsconfig.app.json" },
    { "path": "./tsconfig.node.json" }
  ]
````

```
}
```

frontend/tsconfig.node.json

```
{
  "compilerOptions": {
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.node.tsbuildinfo",
    "target": "es2023",
    "lib": ["ES2023"],
    "module": "esnext",
    "types": ["node"],
    "skipLibCheck": true,

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "moduleDetection": "force",
    "noEmit": true,

    /* Linting */
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["vite.config.ts"]
}
```

frontend/vite.config.ts

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [react()],
  server: {
    host: true, //Thia Exposes the server to the LAN
    port: 5173,
  }
})
```